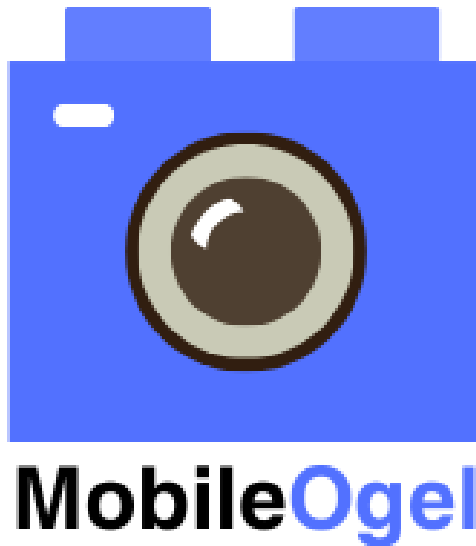


Engineering Project – SYSC 4907

The One and Only MobileOgel



Written by:

Shuvaethy Neill 101143478

Harsimran Kanwar 101143556

Guy Morgenshtern 101151430

Aaron Gabor 101108064

Andre Hazim 101141843

Supervised by:

Dr. Lynn Marshall

Abstract

This project introduces the development of a mobile application, Mobile Ogel, aimed at enhancing the Lego building experience for users of all ages. The application leverages machine learning (ML) and computer vision to revolutionize the Lego experience by accurately identifying and inventorying Lego pieces through the user's device camera. The system generates personalized building instructions for diverse Lego models based on the pieces scanned from an image of the user's unused Lego bricks.

Motivated by the challenges faced by Lego enthusiasts with unused pieces, Mobile Ogel addresses the environmental impact of Lego wastage and fosters creativity. The project integrates modern software development and ML techniques to repurpose leftover Lego collections.

Table of Contents

Chapter 1 Introduction	7
1.1 Problem	7
1.1.1 Problem Statement.....	7
1.1.2 Problem Motivation.....	8
1.1.3 Proposed Solution.....	8
1.2 Background	9
1.2.1 Objectives	9
1.2.2 Current State of the Art	9
1.2.3 Computer Vision.....	9
1.2.4 Architectural Style.....	11
1.2.5 Scope and Limitations	11
1.2.6 Methodology.....	11
1.3 Overview of Remainder of Report	12
Chapter 2 The Engineering Project.....	13
2.1 Health and Safety	13
2.1.1 Data Security and Privacy Measures	13
2.1.2 User-Centric Design	13
2.1.3 Accessibility	14
2.2 Engineering Professionalism.....	14
2.2.1 Ethical Considerations.....	14
2.2.1.1 Data privacy	14
2.2.1.2 Accounting for Machine Learning Bias.....	14
2.2.2 Protecting the Public Interest.....	15
2.2.2.1 Using a Human-Centric Approach.....	15
2.2.2.2 Image Processing Consent.....	15
2.2.3 Innovation and Serving Society.....	15
2.2.3.1 Using Cutting Edge Technology.....	15
2.2.3.2 Reducing Consumer Waste.....	15
2.2.4 Risks and Mitigation Strategies	16
2.2.4.1 Integration Risk.....	16
2.2.4.2 Scope Risk	16
2.2.4.3 Delay Risk.....	16

2.2.4.4 Information Corruption Risk.....	17
2.2.4.5 Model Training Delays	17
2.2.4.6 Accessibility and User Experience Risk	17
2.3 Project Management.....	17
2.3.1 Work Plan	18
2.4 Justification of Suitability for Degree Program	19
2.4.1 Group Qualifications:	19
2.4.2 Andre	20
2.4.3 Harsimran	20
2.4.4 Shuvaethy	21
2.4.5 Guy	21
2.4.6 Aaron	22
2.5 Individual Contributions	22
2.5.1 Project Contributions	22
2.5.2 Report Contributions	24
Chapter 3 Requirements.....	28
3.1 Functional Requirements.....	28
3.2 Non-Functional Requirements	29
Chapter 4 Design.....	31
4.1 Discovering Buildable Lego Use Case.....	31
Chapter 5 Trade-Offs	32
5.1 Mobile vs. Web Application	32
5.1.1 Mobile Application.....	32
5.1.2 Web Application.....	33
5.1.3 Mobile vs Web Application Conclusion	33
5.2 Python vs. C and C#.....	33
5.2.1 Python.....	34
5.2.2 C and C#	34
5.2.3 Python vs. C and C# Conclusion	35
5.3 Relational Databases vs. Non-relational Databases	35
5.3.1 Relational Databases.....	35
5.3.2 Non-Relational Databases	35
5.3.3 Relational Databases vs. Non-Relational Databases Conclusion	36

5.4 YOLO vs. Faster-R-CNN.....	36
5.4.1 YOLO	37
5.4.2 Faster-R-CNN.....	37
5.4.3 YOLO vs. Faster-R-CNN Conclusion.....	37
5.5 Choosing Between Different YOLO Versions.....	38
5.5.1 YOLOv5	38
5.5.2 YOLOv8	38
5.5.3 Choosing Between Different YOLO Versions Conclusion.....	39
5.6 Running Model Inference Locally vs. Server-side.....	39
5.6.1 Local	39
5.6.2 Server-side	40
5.6.3 Running Model Inference Locally vs. Server-side Conclusion.....	40
Chapter 6 Implementation.....	41
6.1 Frontend	41
6.1.1 Current Frontend Features	43
6.1.2 My Pieces Page Display Logic	53
6.1.3 Library Page Display Logic.....	53
6.2 Backend.....	54
6.2.1 Client-Side System	54
6.2.1.1 Inventory Management	54
6.2.2 Server-Side System.....	55
6.2.2.1 Assembly Instruction Management	55
6.3 Machine Learning	55
6.3.1 Model Prototypes.....	56
6.3.2 General Dataset Information	56
6.3.3 Current State of the Model	58
6.3.4 General Model Information	60
6.3.5 Training Techniques	61
6.3.6 Metrics	62
6.3.7 Translation of the Model	67
6.3.8 Colour Detection.....	68
Chapter 7 Testing.....	71
7.1 Manual Testing.....	71

7.2 Unit Testing.....	71
7.3 User Testing	71
7.4 Requirements Testing.....	72
Chapter 8 Conclusion and Reflections.....	73
8.1 Conclusions	73
8.2 Reflections.....	74
8.2.1 Alternative Solutions	74
8.2.2 Project Extension Recommendations	74
8.2.3 Project Positives	75
References.....	76
Appendix 1 Glossary.....	81
Appendix 2 Gantt Chart	85
Appendix 3 Course Codes	86
Appendix 4 Progress Report	87

Chapter 1 Introduction

The purpose of this document is to convey the development of a mobile application that not only enriches the Lego [43] building experience for users of all ages but also preserves the essence of what makes Lego truly timeless. As a team of five engineers, we researched and developed a Lego building application called MobileOgel. This document's contents include the introduction, background, project description, requirements, project relation to our degree program, risk and mitigation strategies, the work plan, implementation, and what was achieved by the end of this project. Technical terms have been italicized throughout the text of this report, with their definitions provided in the glossary section.

1.1 Problem

Lego has been a staple childhood toy since the mid-twentieth century and has since become popular amongst all age groups. As a result, the demand for Lego has increased dramatically year over year. As of 2008, The Lego Group had manufactured over 400 billion Lego pieces [18], and in 2018 reported that over 70 million pieces are sold annually [11].

The large consumption of Lego ultimately leads to an excess of pieces that consumers do not know what to do with. Which in result, creates both environmental issues and missed opportunities for education and enjoyment. Utilization of leftover Lego to prolong its life span is a unique problem that lends itself to solutions which can leverage modern software development and ML techniques.

To fuel new growth, Lego has successfully developed new product lines based on franchises that are popular now. This creates increasingly niche sets comprised of comparably niche Lego pieces. Additionally, each set will come with extra pieces that will not fit on the build. As a result, there are many pieces that go unused and as interest in new sets increases, any existing Lego that may have been passed down gets neglected and eventually thrown away.

The Weather Network predicts that discarded Lego pieces can survive in our oceans for upwards of 1,000 years [11] greatly contributing to ocean pollution that disrupts both animal and human life.

In addition to environmental impact, spare, unused Lego pieces are all missed opportunities for users to exercise their creativity. Lego has been shown to increase children's spatial awareness and problem-solving skills [22][20].

1.1.1 Problem Statement

Lego is a staple that has provided fun for countless people for years and with releasing new sets on a regular basis, it shows no signs of slowing down. Along with Lego's immense popularity, there has come an ever-increasing demand for new pieces, and as a result, a surplus of unused ones. While seeming harmless, this surplus contributes not only to missed opportunities for creativity, but serious environmental challenges, as discarded Lego pieces can survive in oceans for centuries.

The focus of this project is to provide a proactive solution to both these problems, by developing a mobile application called Mobile Ogel. The goal of Mobile Ogel is to provide Lego enthusiasts with an intuitive app to help make the most out of their unused Lego. Mobile Ogel will prompt users to take a photo of a pile of Lego after which all their pieces will be scanned and categorized using computer vision. After inventorying, the users will be provided with a custom list of build instructions for different Lego sets that can completely or close to completely be built with the pieces scanned.

The envisioned app works to bridge the gap between creative engagement with responsible environmental practices by leveraging machine learning and software development techniques. By addressing the issues brought by Lego piece surplus, this project aims to provide a holistic and impactful contribution to both sustainability and the enrichment of users' experience.

1.1.2 Problem Motivation

As a group of Lego fanatics, we have embarked on a journey to find a solution for our ever-growing pile of extra Lego pieces while also putting the skills we have acquired throughout our degree into practice.

In addition, like many people our age, we were inspired by thrifting. The ability to upcycle and breathe new life into things we already own means that we can be sustainable and aim to channel this concept into our project. Not only do we want to revive old Lego but also encourage sustainable practices within our daily lives to bring a new sense of excitement.

1.1.3 Proposed Solution

- **Solution Approach:** Develop a mobile application using ML and computer vision technology to accurately recognize and inventory various Lego pieces through the user's device camera.
- **Application Functionality:** The application will analyze the identified pieces and provide tailored building instructions for different Lego models that users can create using their available pieces.
- **Objective:** To provide alternative build suggestions, reduce Lego waste, and enhance the overall user experience for Lego enthusiasts and educators.

1.2 Background

The 4th year capstone project serves as a platform for graduating students to demonstrate their skills and understanding developed throughout their degree and be able to apply the acquired knowledge to address real-world challenges. In this context, the project focuses on the growing trend of thrifting and reusing old toys, this is especially popular among the younger generation due to its sustainability and cost-effectiveness. The aim to extend similar principles to the realm of Lego enthusiasts who often have an extensive collection of unused Lego pieces. The primary objective is to address the challenge of repurposing these large and neglected sets of Lego, transforming them into valuable resources.

1.2.1 Objectives

The primary objective of our 4th year capstone project was centered around addressing the issue of underutilized Lego collections among enthusiasts. To complete this objective we created a system to both inventory Lego pieces effortlessly and determine Lego builds using those pieces. This was achieved through creating user-friendly interactions within the UI allowing users to capture images and add pieces into their collection easily, detection models and algorithms to count those pieces, databases to store both pieces and Lego sets, and unique matching logic to determine possible Lego sets to build.

1.2.2 Current State of the Art

Modern ML techniques have been used in increasing amounts in everyday life [4]. One such example is computer vision, which is largely used for object detection in areas like facial and pattern recognition [43]. Advancements in the field have made computer vision more precise and more widely available for projects like Mobile Ogel.

While existing products like BrickIt have also used computer vision models to scan and inventory Lego pieces, they often lack simplicity in their design and model accuracy that Mobile Ogel aims to provide [56].

In addition to categorizing and inventorying, this project has a matching component wherein pieces are used to determine which Lego sets can be constructed. This can be done using regular relational databases to store information about existing Lego sets, the number of pieces needed, their colour, and links to their instructions. Afterwards, the issue of determining which sets can be built can be solved mathematically using the following equation:

$$pieces\ required\ for\ given\ set \subseteq inventoried\ pieces$$

Further, techniques like “*Fuzzy Matching*” can be used to determine sets whose pieces may vary slightly from those available [51]. Allowing certain blocks to be substituted for ones of different colours, or situations where a 2x4 block is needed and can be substituted by two 2x2 blocks.

1.2.3 Computer Vision

Computer vision is a rapidly growing field with applications in research and the commercial world.

Computer vision engineers can be thought of as a subset of *deep learning* (DL) and ML engineers [2]. There are many *Artificial Intelligence* (AI) developers that implement computer vision solutions that can identify and classify objects in real time. Image classification can take many forms from face detection, to pose estimation and optical flow.

DL algorithms tend to be very useful when solving computer vision problems. The characteristics of convolutional neural networks enable the detection and extraction of spatial patterns and features that reveal themselves in visual data [2]. The rapid evolution of computer vision is reshaping sectors such as automotive, healthcare, and robotics, making it challenging to keep abreast of the most recent breakthroughs, shifts, and progressions.

There are many technologies that influence computer vision development and shape the way for its future. Some of these technologies are:

- Cloud computing services that help scale DL solutions [2].
- Automated ML solutions that reduce the repetitive work required in a standard ML pipeline [2].
- Transformer architectures developed by researchers that optimize computer vision tasks [2].
- Mobile devices incorporating computer vision technologies [2].

DL methods have historically been deployed for solving computer vision problems, relying on *convolutional neural networks* (CNNs) for tasks such as face detection, lane detection, and pose estimation [2]. However, there is a new paradigm in computer vision that is emerging with the introduction of transformer architectures.

The Transformer is a DL architecture introduced in the paper “Attention is All You Need”. The paper offers a different approach by utilizing the attention mechanism to compute the significance of different parts of input data relative to one another. Unlike traditional CNNs, transformers have gained traction in vision-related tasks, despite their roots in *natural language processing* (NLP). Notable examples of transformer models include Generative Pre-trained Transformer (GPT) and Bidirectional Encoder Representation from Transformer (BERT) [2].

1.2.4 Architectural Style

From a high-level overview, our project contains a Client-Server architecture. The Client-Server model is a *distributed architecture* style that divides the workload between the two entities [44]. A server provides resources or services, and the clients request said services. When a client computer sends a data request to a server via the internet, the server receives and processes the request before returning the desired data packets to the client [44].

There are three different types of a Client-Server architecture: centralized, distributed, and decentralized. In a centralized style architecture, the clients only render results and query the server and the server knows everything and does everything. With a distributed architecture, the client does everything and knows everything while the server is used for communication from client to client. Finally, with a decentralized client server, the clients have their own local data and know how to do tasks with its data and a server knows the data that is common for each client and can do common tasks [44].

MobileOgel makes use of distributed architecture with all model, set matching logic, interaction logic, and rendering done by the iOS application alone and both databases exist only as stores to be queried by the frontend.

1.2.5 Scope and Limitations

The scope of our capstone project is to develop a robust and user-friendly AI-powered system for the identification, categorization, and repurposing of unused Lego pieces. It is important to acknowledge certain limitations of our system. The accuracy of identification and categorization may be influenced by factors such quality of the input image and the diversity of Lego pieces. Additionally, the success of repurposing suggestions depends on the completeness and accuracy of the database. While the system aims to cater to a broad range of Lego enthusiasts, individual preferences may pose challenges in providing universally applicable recommendations. Striking a balance between inclusivity and specificity is a consideration, and the system's effectiveness may vary based on user's preferences. Despite these limitations, we are committed to optimizing the system within the defined scope to deliver a valuable tool for Lego enthusiasts.

1.2.6 Methodology

To develop our AI software, we are using the YOLOv5 model as the core technology for object detection and recognition within the Lego identification system. Our implementation of YOLOv5 will begin with data processing, including image augmentation techniques to enhance the model's ability to generalize across different Lego pieces with various conditions. The model will be trained on various diverse datasets of Lego piece images, capturing various shapes, colors, and sizes. The training process involves optimizing the model's weights and adjusting parameters to achieve high accuracy in Lego piece identification. We will utilize learning by the YOLOv5 model with pre-trained weights on a large dataset, adapting it to our specific Lego dataset during training. The model will then be integrated into the larger system architecture for real-time piece identification. The system's user interface will leverage the predictions from the YOLOv5 model to provide users with an intuitive and interactive experience. Overall, the

YOLOv5 model forms the backbone of our methodology, providing a powerful and efficient solution for Lego piece detection and recognition in real-world images.

In summary, our 4th-year capstone project is a demonstration of acquired skills and knowledge, addressing the contemporary trend of thrifting in the context of Lego enthusiasts. Focused on repurposing underutilized Lego collections, the project uses the YOLOv5 model for efficient object detection and recognition. Our objectives include having a user-friendly interface, identifying and categorizing Lego pieces, and suggesting repurposing options based on available inventory. As the project's scope includes a robust and user-friendly AI-powered system, acknowledging our limitations is important. We plan to provide a valuable tool for Lego enthusiasts, contributing to the reuse of old Lego.

1.3 Overview of Remainder of Report

This report discusses in detail our 4th-year capstone project, which aims to repurpose underutilized Lego collections using an innovated AI-powered system. We will begin by delving into the scope of the project (Chapter 2) which includes health and safety considerations, the importance of engineering professionalism in the development and execution of our project, the project management aspects of our project, justification for the suitability of our project within the context of our degree, along with individual contributions that acknowledge the unique roles and efforts invested by each team member. Next, Chapter 3 discusses functional and non-functional requirements of our application followed by Chapter 4 which outlines our two main use cases for this application. Chapter 5 focuses on the trade-offs we have considered, like using certain frameworks, technologies, and programming languages. Finally, Chapter 6 focuses on the technical aspects of our project, making clear the core functionalities and features of our project such as our frontend, backend, and machine learning parts.

By exploring these sections, we aim to provide a good understanding of our capstone project, encapsulating the many facets that make up the whole project.

Chapter 2 The Engineering Project

This section highlights the meticulous planning and considerations taken throughout the development of our project. Emphasizing health, safety, and user-centric design, this chapter outlines our strategies to ensure user privacy, foster an intuitive experience, and prioritize user well-being. Additionally, it explores the ethical considerations, innovation initiatives, and risk management frameworks integrated into our project. The chapter concludes with insights into our agile project management approach, individual contributions, and a comprehensive justification of our team's qualifications aligning with the project's scope and degree program expectations.

2.1 Health and Safety

Ensuring the safety and well-being of our users has been a key concern in the development of Mobile Ogel. Significant attention has been dedicated to addressing safety and health concerns to create an enjoyable and secure user experience. The following measures have been implemented:

2.1.1 Data Security and Privacy Measures

Photos provided by the user play a crucial role in the LEGO piece identification phase involving our machine learning model. Photos submitted by users are only used to identify LEGO pieces within the application. These photos are processed locally on the user's device and are not transmitted or stored on external servers.

Before the application accesses the device's camera for image capture, users will be explicitly prompted to grant permission. This ensures that users have full control over when and how their camera is used within the application. We are committed to being transparent to users, so they are informed about the specific purposes of camera access and are provided with the option to deny permission. Within the onboarding process, we also emphasize that users should refrain from capturing photos of individuals without their explicit permission. The application is designed and intended to recognize and catalog LEGO pieces, and the unintentionally capturing images of individuals goes against our values of user respect and privacy.

By implementing these security and privacy measures, we aim to provide users with a secure and trustworthy platform for their LEGO building experience.

2.1.2 User-Centric Design

Our application was implemented with an intuitive and simple interface, aiming to minimize confusion or frustration during scanning and inventory management. The user interface only includes the essential elements required to efficiently complete tasks. For instance, the scanning page seamlessly directs users to a live camera view, streamlining the process of capturing LEGO pieces. The overall navigation within the application follows a clear and straightforward structure ensuring that users can easily understand and follow the steps. By prioritizing simplicity, users can focus on their creative process without unnecessary complexities.

2.1.3 Accessibility

This application is also designed with inclusivity in mind, and SwiftUI's built-in accessibility support has been leveraged to enhance the user experience for all individuals, regardless of their abilities. SwiftUI seamlessly provides default accessibility labels and values for UI elements including text and interactive elements such as buttons without the need for a developer to explicitly do so. This allows users relying on screen readers or other assistive technologies to receive meaningful and contextually relevant information as they engage with our application. This approach not only aligns with best practices in inclusive design, but also simplifies the development process by allowing us to prioritize accessibility without sacrificing efficiency [56].

Our commitment to accessibility extends beyond the built-in features of SwiftUI, encompassing a comprehensive approach that aligns with the principles outlined in the W3C Mobile Accessibility Mapping document. Although our application does not require keyboard input, our focus shifted to prioritizing touch target size, spacing, and providing clear indications of actionable elements through conventional shapes, iconography, and colour offsets [57].

2.2 Engineering Professionalism

Throughout our project and the development of this application, we have prioritized our professional responsibilities and approached our tasks with the goal of serving and protecting the public interest. Each milestone of our project was met with discussions on ethics and the interest of our target audience. Below are some considerations we took throughout the development of our project.

2.2.1 Ethical Considerations

In the pursuit of crafting a responsible application that adheres to Engineering Principles outlined by Professional Engineers Ontario (PEO), our development approach for Mobile Ogel extended beyond technical innovation. Ethical considerations served as a foundational pillar guiding our practices and principles. The following sections highlight ethical considerations that were considered.

2.2.1.1 Data privacy

It is important the user is aware of how their data is being used and that we have their consent. As developers, we need to ensure the data being collected follows compliance regulations and is disposed of properly. Also, in our app, the photo captured will not be stored in our system without user approval.

2.2.1.2 Accounting for Machine Learning Bias

Machine Learning models are trained based on the data we feed it. When models are trained on biased data, they can perpetuate throughout the system. Throughout the training process, we made it a priority to train the model with a variety of datasets, including different angles and

lighting for maximized results. Not to mention, we have handpicked our dataset to ensure no sensitive data is present in the background.

2.2.2 Protecting the Public Interest

In our commitment to serve and protect the public interest, we have implemented various processes/checks in place to ensure interest is at the center of every decision being made.

2.2.2.1 Using a Human-Centric Approach

The app we developed will primarily be used by Lego enthusiasts of all ages. Following a user-centric approach allowed us to develop an app that caters to the needs of users, ensuring it is accessible and usable for users of different abilities and backgrounds. As a group, we want our application to be easy to use for everyone. We followed accessibility guidelines for all UI elements including the optimal color scheme to be accessible to those that suffer from color blindness.

2.2.2.2 Image Processing Consent

It is essential for the users to be informed about proper photo-taking etiquette when uploading an image to our system and safeguarding their data. As developers, we can inform the users of the practices we expect users to follow when uploading an image of their Lego pieces. For example, it is important to ensure that no sensitive data or information is visible in the photo. This includes unintentional inclusion of IDs, financial documents, or other identifying information.

Encouraging users to be mindful of their surroundings and allowing them to approve the image before being stored in the system empowers them to take proactive steps in preserving their data security.

2.2.3 Innovation and Serving Society

As aspiring software engineers, we have a duty to serve society in hopes of making it better. The following sections aim to describe how we can serve society through our app.

2.2.3.1 Using Cutting Edge Technology

As aspiring software engineers, we have a duty to serve society in hopes of making it better. We leveraged the latest technology in software development and machine learning to address the problems faced by many Lego enthusiasts. Additionally, using cutting edge technology allowed us to develop more sustainable solutions that are well maintained and long lasting. This helped us to revolutionize the Lego building experience, promoting innovation and creativity within society.

2.2.3.2 Reducing Consumer Waste

Using our app, we aim to bring old and unused Lego pieces back to life. This contributes to the broader societal goal of sustainability by instilling the values of responsible consumption. Our app encourages users to make use of their existing Lego pieces, we minimize plastic waste by

reusing existing pieces. This could inspire users to adopt more environmentally conscious practices in their lives.

2.2.4 Risks and Mitigation Strategies

Throughout this project, we aimed to diagnose the risks surrounding the project and create mitigation strategies to ensure the smooth and correct operation of the application. The following sections will break down some of the risks associated with this project and give mitigation strategies that were employed to prevent or resolve the risk.

2.2.4.1 Integration Risk

Description: The project involves integration between the interface otherwise known as the frontend and the server side otherwise known as the backend for this application. If the interface and server side do not communicate properly, the application will not work.

Mitigation Strategies: Ensured a strong connection between the interface and the server side of the application. Allowed enough time to thoroughly test many different connection attempts to the server side from the interface. Also, we worked with frameworks that have built-in support for integration with these two systems, reducing or removing configuration challenges.

2.2.4.2 Scope Risk

Description: The scope description of the project must be clear, concise and without issue to ensure there are no problems when executing it. Building a project from the ground up makes it vulnerable to scope creep as near the beginning of the project it is more difficult to decide on a final direction.

Mitigation Strategy: Confirmed the project's scope with group members and the faculty adviser before proceeding with its development. The scope was revisited monthly to ensure that the product remains feasible.

2.2.4.3 Delay Risk

Description: Work not being completed according to the scheduled milestones of the project.

Mitigation Strategies: Made sure that everyone was held accountable for their allotment of work and that they gave themselves enough time to complete it. Ensured that the group created a schedule for the completion of work that was compatible with all group members. We also conducted weekly check-ins to keep track of tasks and catch delays early on so they could be addressed. Furthermore, to mitigate running out of time to complete all aspects of the project, all requirements, functional and non-functional, were divided up into essential and non-essential. All essential requirements were implemented first and non-essential requirements only after the application was functional. Some non-essential requirements were: 4.1 #5 tracking colour, 4.2 #3 storing user data, 4.2 #4 handling multiple users, 4.2 #5 ensuring low downtime, 4.2 #7 user interface usability.

2.2.4.4 Information Corruption Risk

Description: When files are stored digitally there is always the possibility that the file can become corrupted, damaged, or lost. This could have caused a large setback and require re-doing already completed work.

Mitigation Strategies: Ensured that proper data protection and storage practices were followed, such as the 3 2 1 data backup strategies [39].

2.2.4.5 Model Training Delays

Description: When training a model, the time it takes can vary from 24h to several weeks. The time it takes is dependent on the platform being used and the resources available. For example, Google's free model training provides limited resources, slowing down model training. This caused slight delays to our project.

Mitigation Strategies: The training phase of the model started early with the free membership provided by Google Collab. The team then purchased a Google Collab Pro membership to accelerate training capabilities and mitigate any further delays.

2.2.4.6 Accessibility and User Experience Risk

Description: When creating a mobile app, what may be easy to implement might not always be the best design choice when it comes to user accessibility and user satisfaction. This approach may lead to an interface that is confusing and potentially inaccessible to certain user groups.

Mitigation: Implemented a user-centric approach, created intuitive UI/UX designs, conducted user feedback sessions, and implemented instructions for first time users.

2.3 Project Management

The development of our project followed principles of agile methodology, specifically combining the structured framework of Scrum with the visual adaptability of Kanban to ensure efficient coordination, management, and execution.

Group members met twice weekly, once to discuss progress amongst ourselves where we engaged in Scrum-inspired practices such as sprint planning, retrospectives, and standup meetings. We met once more with our project supervisor to provide updates and seek guidance on any blockers we may have had. Each project milestone represented a sprint, with a curated product backlog outlining the tasks/features required for the completion of our application [7]. To enhance task visibility, we had a Kanban board in place. This visual tool served as a dynamic representation of our project tasks, showing their status and updates in real time [6]. It enabled the team to track progress, identify bottlenecks, and prioritize tasks effectively. Complementing these practices, our group had a dedicated group chat to always keep lines of communication open. It helped to ensure that deadlines were met on time, updates were shared promptly, and potential challenges were addressed collectively.

2.3.1 Work Plan

The successful completion of this project was measured through the achievement of the milestones shown below. Each milestone represents a critical phase in the development process, ensuring that we are on track to meet the project's objectives. This workplan was meant to provide a higher-level overview of the work that needed to be done, more in-depth project tasks and their timelines are defined in the Gantt chart (Appendix 2).

Milestone 0: Project Initiation - *Oct 2-9, 2023 (COMPLETE)*

Subtasks:

- 0.1. Create a project plan and finalize schedule.
- 0.2. Create Github repository.

Milestone 1: Application Setup - *Oct 2-23, 2023 (COMPLETE)*

Subtasks:

- 1.1. Set up the development environment (IDE to be used) for frontend, middleware, and backend development.
- 1.2. Create a basic skeleton for the application, server, and backend.

Milestone 2: Client Mobile Application Development - *Oct 9, 2023 - Mar 2, 2024 (COMPLETE)*

Subtasks:

- 2.1. Design the user-friendly mobile application interface for Lego piece scanning.
- 2.2. Implement the mobile camera functionality for scanning Lego pieces.
- 2.3. Add other necessary features.

Milestone 3: Computer Vision and Model Training - *Oct 2, 2023 – Mar 6, 2024 (COMPLETE)*

Subtasks:

- 3.1. Finalize Lego dataset.
- 3.2. Leverage YOLOv5 to help the application identify Lego pieces from images.
- 3.3. Train ML models to recognize different Lego pieces effectively.
- 3.4. Implement a way to store the recognized Lego pieces in the Inventory Management database.

Milestone 4: Client Inventory Management - *Oct 2, 2023 – Jan 14, 2024 (COMPLETE)*

Subtasks:

- 4.1 Develop the client-side database to store users' Lego collection information (e.g., color, piece name, quantity).
- 4.2. Design database schemas for storing Lego piece inventory.
- 4.3 Create API endpoints to send inventory data to server.

Milestone 5: Server Development - *Oct 2, 2023 – Jan 14, 2024 (COMPLETE)*

Subtasks:

- 5.1. Create a server-side database to store different builds with assembly instructions.
- 5.2. Design database schema for storing Lego assembly instructions.
- 5.3. Implement API endpoints to send data back to the client.
- 5.4. Establish a connection with client.

Milestone 6: User Testing and Accessibility - *Mar 1, 2024 – Mar 22, 2024*
(COMPLETE)

Subtasks:

- 6.1. Conduct user testing on the application's usability and performance.
- 6.2. Make necessary adjustments to enhance the user experience.
- 6.3. Ensure the application follows industry standards for accessibility to accommodate users with diverse needs.

Milestone 7: Documentation and Deployment - *Mar 11, 2024 – Mar 22, 2024*
(COMPLETE)

Subtasks:

- 7.1. Prepare comprehensive documentation, including user guides and developer documentation.
- 7.2. Deploy the application to a production environment or relevant app stores.
- 7.3. Monitor and maintain the deployed application for updates and improvements.

Project Deliverables:

October 30st, 2023 – Project Proposal Due (COMPLETE)

December 8th, 2023 – Oral Presentation and Progress Report Due (COMPLETE)

January 22-26th, 2023 – Oral Presentations (COMPLETE)

March 14th, 2024 – Final Report Draft Due (COMPLETE)

March 22nd, 2024 – Poster Fair (COMPLETE)

April 10th, 2024 – Final Report and Video Submission (COMPLETE)

2.4 Justification of Suitability for Degree Program

Our group is comprised of five software engineering students. The following sections provide an overview of our collective qualifications as a team, complemented by our individual achievements that help diversify our skillset, overall strengthening our project.

2.4.1 Group Qualifications:

Throughout our degree, we have gained a diverse skillset from the courses we have taken along with gained industry experience. From gathering requirements to architectural design, robust coding, database management, and beyond, our degree thus far and industry experiences have prepared us with the knowledge to succeed in our capstone project, aligning us with engineering professional practice standards. All group members have honed their skillset for this project through the completion of key courses such as *SYSC 2100*, *SYSC 4120*, *COMP 3005*, and *SYSC 4106*. These courses have provided us with a solid foundation, ensuring that there will not be a steep learning curve as we embark on this project. Our commitment to continuous learning through this project by taking a ML course (*SYSC 4415*) alongside the project will be an

invaluable asset for our success. In addition to mandatory coursework, we all have experience developing computer vision or ML applications either through industry experience, personal projects, or electives. Through these we have gained insights into the warning signs of project delays, how to address them, and the ability to combine different management techniques for successful project completion, as well as building large scale projects while following best practices and industry standards. Based on this, we are confident that we have the necessary qualifications to succeed.

2.4.2 Andre

During my degree, I have learnt many important things about software development. At the start of the software development lifecycle, I learned how to properly elicit software requirements from *SYSC 3120*. This plays a crucial role in gathering and analyzing requirements ensuring that the project's foundation is well-structured. *SYSC 2004* and *SYSC 3110* taught me fundamental principles to object oriented development and the principles of structuring the codebase around objects and encapsulation. This helped ensure that the projects create adheres to principles of software engineering. *SYSC 4120* taught me the different types of architectures and when to apply them in a software development setting. I was able to apply software engineering principles to create robust and scalable architecture. Furthermore, my experiences in *COMP 3005* equipped me with the skills and knowledge of establishing relational databases and query effectively. All these acquired skills will prove to be useful when configuring the database for our application, creating requirements, and picking an architectural style. This will significantly reduce the learning curve needed for this task. In the upcoming term, I will be taking a ML course (*SYSC 4415*) and a human computer interaction course (*SYSC 4130*). While not specific to software engineering, these emerging technologies are increasingly integrated into software applications. These courses will help me with working with the AI portion of our project when it comes to Lego detection. Finally, *ECOR 4995* taught me the skills of an engineer in professional practice. This course taught me insight into a professional engineer's ethics, practices, health and safety protocols, and so on. This profound understanding of engineering principles will play an important role in upholding the standards and practices throughout the project's development. I believe these skills are heavily related to the project because this is a software-based project, and all these skills are needed for the development of this project. Being able to understand and plan out the software is a much-needed skill in development as I have learned through many different co-ops and school projects. I will be able to demonstrate these skills from the very beginning when we are planning on what architecture we are going to use up until implementing the computer vision into the program.

2.4.3 Harsimran

Throughout my degree, I have learned the importance of the software development life cycle from writing well refined requirements (*SYSC 3120*) to executing these requirements properly to deliver what the clients want (*SYSC 4106*). Additionally, *SYSC 4120* taught me the importance of having an architecture in place along with the impact of certain tradeoffs. This knowledge will come in handy when deciding what our application's architecture should look like and what tradeoffs can be made to deliver on time. Not to mention, my learnings from *SYSC 2100* taught me how to write algorithms that are bulletproof and resistant to unexpected inputs. Moreover, my

learnings in COMP 3005 taught me how to properly set up relational databases and query them. These skills will come in handy when setting up our database for the application and will result in a minimal learning curve. Although not directly relevant, the skills I learned in Professional Practice (ECOR4995) will always be invaluable. This course emphasized the importance of creating a safe work environment, along with how to resolve conflicts amicably should they arise. Throughout this project, I aim to create a safe work environment for everyone so they will feel comfortable sharing their ideas and collaborating. Next semester, I will further improve upon my skillset by taking SYSC 4415 and *COMP 3008*. SYSC 4415 has a focus in machine learning and COMP 3008 has a focus in human computer interaction. I am hoping from these courses, I can learn skills to further improve upon our computer vision model and develop our user facing application in a way that is accessible and easy to understand for all users. From this project, I hope to further develop my programming skills, understand computer vision frameworks, learn about mobile application development/deployment, and how to leverage my skills to solve a real-world problem.

2.4.4 Shuvaethy

Different aspects of this project directly apply to the core principles of my software engineering degree. Through courses such as SYSC 3120, I learned how to write well-defined software requirements and navigate the software development lifecycle. This plays an essential role in outlining this application's functionalities. SYSC 2100 provided me with a solid foundation in algorithms and data structures which will be necessary for effective piece scanning and classification. Through COMP 3005, I gained knowledge in database management and SQL queries. These skills will come into use when designing and managing our project's database, especially for robust inventory management. Furthermore, my learnings from SYSC 4120 will help in choosing appropriate design patterns, balancing important trade-offs, and building a well-structured, scalable application. Additionally, having completed ECOR 4995, I gained valuable insights into conflict resolution, creating a safe work environment, and sustainable development principles. An understanding of these engineering principles will be integral in maintaining high standards and practices throughout the development of this project. Next term, I am also taking an ML (SYSC 4415) and a human-computer interaction course (SYSC 4130). These courses will help me strengthen my contributions to the ML aspects of the project and also ensure we have a user-friendly interface. With the skills I have acquired throughout my degree and the mobile development experience I have gained through co-op, I look forward to the development of this project and to further refine my abilities.

2.4.5 Guy

This project is very multifaceted. Building it from the ground up allows me to exercise not only the programming skills I have learned over my degree, but also software engineering skills. Several elements of this project draw upon the knowledge I acquired in courses such as SYSC 4106 (The Software Economy and Project Management) and SYSC 3120 (Software Requirements Engineering), enabling me to adeptly gather project requirements, strategize for deadlines and milestones, and effectively document the project's progression. SYSC 4120 – Software Architecture and Design equipped me with the capability to determine optimal architectural patterns for the project and pinpoint the project's most important characteristics.

As with any large-scale project, its success relies heavily on good testing. Taking *SYSC 4101* this semester has begun to teach me generic testing techniques that still result in thorough and effective tests that I can apply to any aspect of this project.

On the technical side, *SYSC 2100* taught me both Python fundamentals and how to determine which data structures are more effective in certain situations. This makes me more confident that the choices I make during development keep the application reliable and efficient. This project will also require database design and development, which are both skills I learned in *COMP 3005*. *SYSC 2004*, *SYSC 3110*, and *SYSC 4806* all provided a solid foundation for large-scale project development using object-oriented principles, which I plan to use during development. *SYSC 4415* which I will take next semester will also solidify my ML skills allowing me to strengthen the computer vision portions of this project. *SYSC 4130* will also imbue me with the toolset to create an application that is more user friendly and secure, which I will use to help develop the UI.

Overall, the development cycle of this project encapsulates all aspects of my degree, from planning and design to development and deployment.

2.4.6 Aaron

This project encompasses multiple facets that intersect with various aspects of my degree. The fundamental programming principles acquired in *SYSC 2006* and *SYSC 2004* serve as the foundation of all programming within the project. Additionally, it extends to encompass more specialized concepts expounded in courses such as *SYSC 2100*, *SYSC 2320*, *SYSC 3110*, *SYSC 3120*, *SYSC 3310*, *SYSC 4106*, *SYSC 4504*, *COMP 4102*, and *COMP 3005*.

Notably, *COMP 3005* and *SYSC 4504* have offered valuable insights into database systems and their implementation, which should prove instrumental in the database-related components of this project. The database experience enhanced my appley to properly implement the two databases used in this application. By providing the knowledge necessary to create the databases and implement the querying for each database. However, *COMP 4102* should play a pivotal role in the application of computer vision techniques.

The remaining SYSC courses have contributed significantly to fostering a comprehensive understanding of programming intricacies and diverse methodologies essential for the successful development and execution of a project of this nature. Furthermore, *CCDP 2100* is poised to provide invaluable guidance in the composition and review of reports and presentations associated with this project.

2.5 Individual Contributions

2.5.1 Project Contributions

It is important to us as a team that each of us is exposed to every facet of the project in at least some way. Each of us will contribute to frontend, backend, database, and training/inference pipeline development in some capacity to ensure that we all gain as much experience from this

project as possible. However, as we have our own strengths and expertise, we will focus more on areas that align with our individual strengths. To maintain equal work distribution, we will carefully balance our assignments. The following are our current interests and potential areas of focus for the project:

Table 1: Individual Project Contributions

<u>Name</u>	<u>Contributions</u>
Shuvaethy	Data annotation and classification Swift application development Designing and implementing UI Local database integration and query logic Integrating the model into the application
Harsimran	Swift application development Designing and implementing UI Local and server database integration and query logic Model translation (YOLOv5 to CoreML) and integration into application
Guy	Model training and testing Data pipeline engineering Colour detection algorithm Dataset creation Swift application development Designing and implementing UI Integrating the model into the application
Aaron	Local database creation Server database creation Local and server database integration and querying Data annotation and classification Created, formatted, and edited the 3-minute video
Andre	Model training and testing Dataset creation Data annotation and classification Swift application design

2.5.2 Report Contributions

In our collaborative effort to create the reports for the project, every member of the team played a crucial role in shaping each section. While we shared responsibility, each of us had a specific section in which we made significant contributions. Our individual major contributions to the reports are listed below with links to their respective sections:

Table 2: Group Member Report Contributions

Group Member	Proposal	Progress Report	Final Report
Harsimran	• Introduction • Background • Work Plan • Risks and Mitigation • Relation of Project to Degree • Tradeoffs	• Introduction • Problem Statement • Engineering Professionalism • Justification of Suitability for Degree Program • Frontend • Risks+Mitigation	• 2.2 Engineering Professionalism • 2.3 Project Management • 2.5 Individual Contributions • 6.1 Frontend • 6.2 Backend • 7.1 Manual Testing • 7.3 User Testing • 8.1: Conclusions • 8.2: Reflections
Shuvaethy	• Introduction • Project Description • Tradeoffs • Non-functional requirements • Relation of Project to Degree	• Health and Safety Considerations • Project Management • Justification of Suitability for Degree Program • Frontend	• 2.1: Health and Safety • 2.2: Engineering Professionalism • 2.3: Project Management • 2.4: Justification of Suitability for Degree Program • 2.5: Individual Contributions • 3.2: Non-Functional Requirements • 5.0: Tradeoffs • 6.1: Implementation – Frontend • 7.2: Unit Testing • 7.4 Requirements Testing • 8.1: Conclusions

Guy	• Introduction • Background • Tradeoffs • Work Plan • Relation of Project to Degree	• Problem Statement • Justification of Suitability for Degree Program • Machine Learning	• 2.2: Engineering Professionalism • 2.3: Project Management • 2.4: Justification of Suitability for Degree Program • 2.5: Individual Contributions • 3.2: Non-Functional Requirements • 4: Use Case Diagram • 6.1: Implementation - Frontend • 6.3: Implementation - Machine Learning • 7.4 Requirements Testing
Aaron	• Background • Use Cases • Risks and Mitigation • Relation of Project to Degree	• Justification of Suitability for Degree Program • Backend	• 1.0: Introduction • 2.4.6: Aaron • 2.5: Individual Contributions • 4.1: Use Case • 5.3: Relational Databases vs. Non-relational Databases • 6.2: Backend • 7.2: Unit Test • 8.2: Reflections • Reference

Andre	• Background • Tradeoffs • Functional Requirements • Relation of Project to Degree	• Background • Overview of remainder of report • Justification of Suitability for Degree Program	• 1.1: Problem • 1.2: Background • 1.3: Overview of Remainder of Report • 2.2: Engineering Professionalism • 2.3: Project Management • 2.4: Justification of Suitability for Degree Program • 2.5: Individual Contributions • 3.1: Functional Requirements • 3.2: Non-Functional Requirements • 5.0: Tradeoffs • 6.3: Implementation - Machine Learning • 8.2: Reflections
-------	---	--	---

Chapter 3 Requirements

This section outlines the functional and non-functional requirements that will guide our project towards its objectives. These requirements have been carefully defined to address key functionalities, performance standards, and user expectations.

3.1 Functional Requirements

1. The system shall successfully access the camera of the device being used.
 - Measurement: Test the application on various devices and platforms to confirm access and functionality.
2. The system shall use computer vision to detect the types of Lego pieces.
 - Measurement: Test the application's detection capabilities on a variety of Lego pieces. Assessing the false positives and false negatives and comparing these results to expected accuracy.
3. The system needs to keep track of Lego pieces that have been scanned and recognized.
 - Measurement: Test the application's ability to maintain an accurate inventory over time. This also includes adding and removing pieces as they are scanned.
4. Displays step-by-step instructions on how to build the Lego model.
 - Measurement: Through feedback from users of the application assessing the ease of following instructions.
5. The application should integrate with the database of Lego pieces keeping track of the color, shape, and name.
 - Measurement: Cross-reference information retrieved from the database with known Lego pieces to validate the correctness.
6. Will display the detected bricks in a list with how many of each type of bricks are detected.
 - Measurement: Compare the application's numbers in the database with the physically scanned pieces to ensure accuracy.

7. The system must only allow users to see buildable models.

- Measurement: Test the application's restriction methods to verify that only buildable options are available.

The functional requirements that are crucial to the success of our application are requirements 1, 2, 3, 4, 5, and 7. These requirements are the backbone of our project that must be implemented to have a functional application. The rest of the requirements are still useful and important for the application, but the application can still function without them. Our non-priority requirements will only be ignored if we fail to comply with our scheduled timeframe.

3.2 Non-Functional Requirements

1. The system shall accurately identify Lego pieces to ensure correct suggestions.

- Measurement: Conduct tests to evaluate the accuracy of piece detection and compare it against predefined accuracy threshold of 90%.

2. The system shall have an average response time of 15 seconds and a worst-case response time of 2 minutes when generating tailored build suggestions.¹

- Measurement: Conduct load and responsiveness testing to evaluate the application's performance under different levels of detected pieces.

3. Average application responsiveness shall be within 250ms of user input [15].

- Measurement: Conduct responsiveness testing for each feature under expected application conditions.

¹ The response time of build suggestions was determined by several factors including:

- Average inference time of a YOLOv5s model on the most popular iPhone model as of 2023 (iPhone 11) [16]
- kNN and colour sampling time based on research and preliminary benchmarks of our system [1]
- Average MongoDB query speed for a database with a row for every Lego set [14]
- Expected user response times and iOS development guidelines [15]

4. The system shall securely store user data (including images and personal information) and inform the user of how their data is going to be used.
 - Measurement: Ensure compliance with data protection regulations and inform users of our data protection policy in terms and conditions.
5. The system should be designed to accommodate a growing user base without sacrificing performance.
 - Measurement: Conduct load testing with an increasing number of mock users to ensure the system is still responsive and stable under load.
6. The system shall be available and reliable, with minimal downtime for maintenance and updates.
 - Measurement: Track any incidents of unplanned downtime and conduct regular maintenance activities during off-peak hours to minimize disruption to users.
7. The application shall have error handling capabilities and provide informative error messages.
 - Measurement: Evaluate the clarity and effectiveness of error messages through user testing. Monitor the frequency of reported errors and assess the system's ability to recover from unexpected events.
8. The user interface should be intuitive and user-friendly, ensuring a positive and seamless user experience.
 - Measurement: Conduct usability testing and gather user feedback to assess how easily users can navigate through the application and perform tasks.

The non-functional requirements critical to the success of our application are requirements 1, 2, 4, 5, and 7. Accurate piece identification, responsive build suggestions, data security and privacy measures, scalable performance, effective error handling, and intuitive user interface design ensures optimal user experience and trust.

Chapter 4 Design

4.1 Discovering Buildable Lego Use Case

Description: Use case for Mobile Ogel Application

Primary Actor: User

Secondary Actor: Database

Preconditions:

- Requires the application to be installed.
- Requires connection to the internet.

Use Cases:

- Taking photos with the camera
- Viewing the inventory of scanned Lego pieces
- Editing the inventory of scanned Lego pieces
- Viewing the available sets that the user can build.
- Generating possible sets that can be built.

Relationships:

- Edit Inventory includes view inventory.
- View Available Sets includes Generate Possible Sets
- User is the actor for Take Photo, View Inventory, Edit Inventory, and View Available Sets
- Database is the actor for Generate Possible Sets

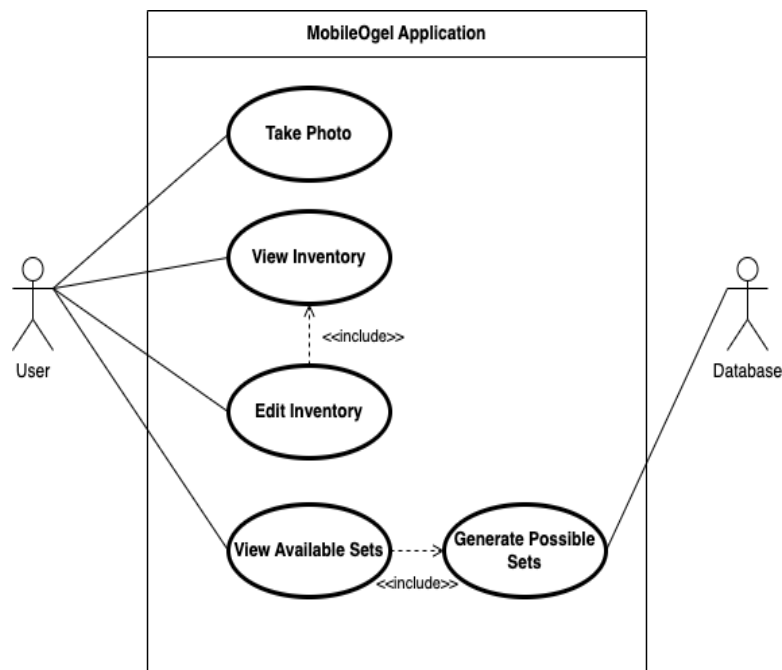


Figure 1: Scanning Lego Use Case

Chapter 5 Trade-Offs

The following section discusses several trade-offs and design decisions we have considered that best fit our use cases. It includes discussions on various application types (mobile vs web, python vs C++), various training frameworks (Faster-R-CNN vs YOLO), different YOLO versions (v5 vs v8), and discusses tradeoff related to certain programming languages, to name a few.

5.1 Mobile vs. Web Application

5.1.1 Mobile Application

Table 3: Pros and Cons of a Mobile Application

Pros	Cons
- Access to device hardware is significantly easier and faster since you are working with the operating system of the device. - Native mobile applications offer better performance when compared to a cross-platform solution because they can be optimized for the specific hardware of the device. - Offer an extensive set of <i>software development kits</i> (SDKs) and <i>application programming interfaces</i> (APIs), allowing developers to access device specific features and functionalities.	- Will only allow us to create an application for one operating system at a time. - Necessary to follow standards for designing an application for a specific operating system.

5.1.2 Web Application

Table 4: Pros and Cons of a Web Application

Pros	Cons
- Can be easily accessed using a web browser. - Can save information on the cloud rather than saving it on the users' device, allowing them to use multiple devices.	- Cannot access the devices' hardware easily. - Performance will be a factor since we would be running the application on a web browser. - Dependency on an internet connection

5.1.3 Mobile vs Web Application Conclusion

For our project, we will choose to create a mobile application because it is more tailored to our needs. We need easy access to the devices' hardware, to be able to save information locally, as well as having the computer vision algorithm run from the devices' hardware components rather than on the server side.

5.2 Python vs. C and C#

For the ML and computer vision aspects of this project, we considered using programming languages such as Python, C, and C#. Each language has its own set of strengths and weaknesses that have been outlined below.

5.2.1 Python

Table 5: Pros and Cons of using Python

Pros	Cons
- Widely used for ML and computer vision purposes and has an extensive collection of libraries and <i>frameworks</i> tailored for these topics such as OpenCV, TensorFlow, and PyTorch. - Libraries provide a pre-trained model to use as a starting point and can be further trained on a specific dataset to adapt them to a particular application. - Clean and readable syntax is helpful for rapid prototyping and iterative model development. - It has an extensive community and well-documented resources for solving challenges related to ML and computer vision.	- Slower compared to compiled languages for performance intensive tasks.

5.2.2 C and C#

Table 6: Pros and Cons of using C or C#

Pros	Cons
- Compiled languages are faster compared to an interpreted language like Python for performance-intensive tasks. - Provide capabilities for manual memory management, which allows for precise control over memory resources.	- Writing and debugging in these languages typically require more effort and time (negative impact on speed of development) compared to higher-level languages like Python. - Precise memory management can be challenging and prone to errors, potentially leading to memory-related issues.

5.2.3 Python vs. C and C# Conclusion

Ultimately, the simplicity, extensive library ecosystem, and widespread use of Python in the ML and computer vision domains make it the most beneficial choice for our project.

5.3 Relational Databases vs. Non-relational Databases

5.3.1 Relational Databases

Table 7: Pros and Cons of using Relational Database

Pros	Cons
- Excels in handling structured data with defined schemas. - Has limitless indexing capabilities, which results in faster query response time. - SQL support. - Enforces data integrity with primary/foreign key relationships.	- Less flexibility with unstructured data. - Scaling horizontally may be challenging.

5.3.2 Non-Relational Databases

Table 8: Pros and Cons of using Non-Relational Databases

Pros	Cons
- Offers flexibility and scalability since you are not restricted to a schema. - Better at handling high traffic loads than SQL. - Many NoSQL databases provide built-in features for availability and fault tolerance. - NoSQL databases are often more cost-effective when operated at scale.	- Lack in structure can make it difficult to maintain the integrity of data since it does not use a tabular schema or enforce data constraints. - Does not support complex relationships and transactions between data entities. - Tools and ecosystem around NoSQL databases are often less mature than SQL and have fewer available tools and libraries.

5.3.3 Relational Databases vs. Non-Relational Databases Conclusion

As a group, it was decided to use both relational and non-relational databases for our two databases. For the client-side database, a relational database was chosen as the Lego piece information to be stored in the database has clearly defined relationships and lends itself best to a set schema. It has capabilities to create, retrieve, update, and delete records and heavily relies on primary/foreign key relationships to identify related data across multiple tables. By leveraging a relational database, we prioritize the accuracy and consistency of data, enabling a seamless user experience while sacrificing some of the flexibility that non-relational databases might provide [40].

For the server-side database, a non-relational database was chosen as the majority of the group had more experience with using non-relational databases for storing information on a remote server. After the oral presentation and reviewing the second reader Professor Esfandiari's feedback, the group decided to continue with the non-relational database for the server-side database. The decision was made to continue with the non-relational database after considering the complexity of creating a structured schema for the Lego sets where the number of different pieces that are used in each set and the types of pieces used can change drastically. While relational databases allow for more complex queries, the combination of time constraints and the complexity of querying the "fuzzy matches" without multiple connections to the server brought the group to conclude that the use of the non-relational database would allow for a better more seamless integration with the object-based programming on the client side. Compared to the higher amount of processing on the received information when a relational database is used. In addition, non-relational databases allow for better horizontal scaling compared to most relational databases which would allow for more servers to be added to distribute the possible load of multiple users if the application were to ever be released.

5.4 YOLO vs. Faster-R-CNN

YOLO and Faster-R-CNN (R-CNN) are both popular frameworks in the industry and could both be successful in detecting Lego pieces. Their differences lie in their architecture. R-CNN uses 2-stage architecture where inference is divided into *region proposal generation* and then *object classification* [44]. Comparatively, YOLO uses 1-stage architecture, classifying regions and detecting objects in one swoop through the image. This makes R-CNN potentially more accurate than YOLO. The tradeoff here is accuracy versus efficiency and trying to maximize model performance and user satisfaction while minimizing inference time and resource use.

5.4.1 YOLO

Table 9: Pros and Cons of using YOLO

Pros	Cons
- Known in industry for real-time processing speed. - Optimized for near real-time detection. - YOLO variants offer models that can run more efficiently on smaller devices. - Accuracy concerns can be addressed through rigorous and intentional training. - Extensive support from PyTorch and OpenCV - Group experience in using the model	- Potentially less accurate than Faster R-CNN - Some reports show it is difficult to detect small objects that appear in groups

5.4.2 Faster-R-CNN

Table 10: Pros and Cons of using Faster-R-CNN

Pros	Cons
- Potential for higher accuracy for small objects compared to YOLO. - General accuracy is higher on average compared to YOLO. - Higher localization due to 2-stage architecture	- More resource intensive than YOLO - Slower inference due to 2-stage architecture - Larger learning curve

5.4.3 YOLO vs. Faster-R-CNN Conclusion

To summarize, we would like to use a model that promotes simplicity, has real-time processing capabilities for faster results, and is not resource intensive on devices. Knowing this, YOLO is the more suitable choice.

5.5 Choosing Between Different YOLO Versions

YOLO is a general-purpose object detection model that has many different versions and a simple API. Compared to other models, training is straightforward and the model itself is flexible allowing it to be trained effectively on custom datasets. YOLO is widely regarded as one of the best real time object detection models.

There are several different YOLO versions. YOLOv5 and YOLOv8 are two that have performed very well on PC and devices with *Graphical Processing Units (GPUs)*. Both have different versions at different sizes/parameter count.

5.5.1 YOLOv5

Table 11: Pros and Cons of using YOLOv5

Pros	Cons
- More research in performance since it has been out for a while. - Reliable. - Nano size of the model has been proven to run reliably and efficiently on mobile and non-GPU devices. - The team has experience training and building applications around YOLOv5.	- Accuracy and inference speed metrics are lower than v8 on larger sizes of the model.

5.5.2 YOLOv8

Table 12: Pros and Cons of using YOLOv8

Pros	Cons
- Latest YOLO model. - Comprehensive framework that simplifies testing and image labelling. - Good midpoint – YOLOv8 Nano has more parameters than YOLOv5 Nano but less than YOLOv5 Small. - iPhones have a GPU so it is possible YOLOv5 will run nicely.	- Slow training. - Lower FPS on inference compared to YOLOv5. - Unclear performance on non-GPU devices. - Even though iPhones have GPU, it is unclear if it is strong enough to run v8.

5.5.3 Choosing Between Different YOLO Versions Conclusion

YOLOv5 has been proven to run reliably on mobile devices that are now a few generations old. By using this model, our application will have a better chance at being compatible with more devices than YOLOv8. In addition, the faster training of YOLOv5 allows us to incrementally improve the model quickly and experiment with tweaking different parameters. Finally, as some team members have experience with the YOLOv5 framework, there will be less of a learning curve, allowing for faster development.

5.6 Running Model Inference Locally vs. Server-side

Model inference can either run locally on the users' machine, or server-side. There exist many examples of successful applications running locally and others successfully running server-side. Inference speed, hardware compatibility, cost, and our most important functional and non-functional requirements are crucial factors to consider when making this decision.

5.6.1 Local

Table 13: Pros and Cons of Local Inference

Pros	Cons
- Inventorying can be done locally, meaning a portion of the application can be run offline. - Running inference locally requires less application-server communication. Data transfer is numerical information instead of transferring entire images. - Lower latency as inference does not depend on the server. - Allows for more seamless real-time inference, something YOLOv5 is very good at	- Updating the model is more challenging and would require pushing an update to the application rather than updating the server. - Device compatibility: Older or budget models may not have the resources to run inference. - Quantizing the model or using smaller versions of YOLOv5 reduces accuracy of the model. - Inference speed itself could be slower (although this is counteracted by not needing to send server requests) - Application size would be larger as the model and any additional software needed to run inference would be packaged with the application.

5.6.2 Server-side

Table 14: Pros and Cons of Server-side Inference

Pros	Cons
- Inference architecture would mimic <i>SOA</i>, allowing for model updates to be done in one spot. - Hardware compatibility issues disappear. - Reduced application size compared to local inference. - Larger and more accurate models could be used.	- High latency due to large server requests - Inference would have to rely on static images, resulting in less real-time capabilities. - Entirety of application would be dependent on the internet. - Sending large images to the server repeatedly is costly for users on data plans. - Running complicated inference requires more server up time which is costly. - Dataflow becomes strange - o Images scanned locally --> sent to server --> server runs inference to inventory Lego pieces --> Sends inference results back to user --> Sends available sets back to user

5.6.3 Running Model Inference Locally vs. Server-side Conclusion

Ultimately, high performance, real-time capabilities, and low costs are the most important factors for this portion of the project while low application size and older model hardware compatibility is not crucial. It is important that we create the most efficient dataflow pipeline possible to ensure that our costs stay low, while also ensuring that the user has access to the most accurate scanning we can provide. In addition to this YOLOv5 is inherently compatible with the hardware in iPhones and most Android devices from 2019 at least [44]. The Nano (v5n) and Small (v5s) versions of the model have both been extensively utilized in computer vision applications on mobile devices and have a track record of providing fast and accurate inference, in real-time. For this reason, local inference is the most suitable choice for this project.

Chapter 6 Implementation

The following section provides some insight into the architecture, functionalities and underlying technologies that power our solution, including the frontend, model, and backend. It captures the work our team has accomplished thus far and the improvements that will be made as we carry this project forward post-graduation.

6.1 Frontend

The frontend of Mobile Ogel is designed to provide an intuitive and seamless experience for users, catering to both first-time and returning users. The interface serves to allow a user to scan Lego pieces, view build suggestions, and manage inventories.

The initial flow for new users ensures a straightforward onboarding process. Upon opening the application users will see a splash screen with our application logo before being presented with an instruction page indicating to lay their LEGO pieces on a flat surface and to take a photo from a birds eye view. To maintain data security and privacy, a disclosure at the bottom of the instruction page informs users about the usage of their captured photos and emphasizes not taking pictures of individuals without their consent. Upon pressing the “Get Started” button, users are prompted to grant camera access. If accepted, they proceed to the live camera view, otherwise they are brought to an error page advising them to enable camera access in their device settings for optimal use of the application.

After capturing a photo, users will be brought to a confirmation screen showing them the captured image with two buttons at the bottom. The retake button will allow them to redo the previous step and recapture the image. The proceed button will use the image shown and bring them to the next step.

If the user is satisfied with the photo they took and choose to proceed, a loading screen will appear while the image is processed by the machine learning model. Once the piece detection is complete, the user will automatically be navigated to the ‘My Pieces’ page with the results displayed. Here, users can see a list of detected LEGO pieces with their image, name, and the detected quantity the user has. They are also given the ability to edit their inventory of pieces. There is an ‘Edit’ button near the top of the screen that will bring the user into an editing mode. At the end of the ‘My Pieces’ list, users can click ‘Scan Summary’ to view a page showing the detections made by the model. This page shows the user’s captured image with bounding boxes overlayed on every piece the model detected. By tapping a given bounding box, the user can see the name of the piece detected. This action is togglable, by tapping again, the text is removed. Users can return to the ‘My Pieces’ page by clicking the ‘My Pieces’ button on the bottom of the page. The ‘Scan Summary’ page is meant to work in conjunction with the ‘Edit’ functionality by allowing users to see what the model detected before adding new pieces, avoiding accidental duplication.

Users can also navigate to the ‘Library’ page from here to see what sets they can build by clicking the “See Build Options” button.

The ‘Library’ page has three filters and offers users the ability to explore sets based on perfect matches, similar matches, or a combination of both. A perfect match set is one that the user has all the pieces for, and it will display the image of the build set, its name, the total number of pieces, and a link redirecting to the official step-by-step LEGO instructions via Safari. Fuzzy matches are sets that the user has either all the pieces in the wrong colours or missing at most 3 pieces. The same information will be shown for similar match sets, except these will also show the user which pieces they are missing (Figure 5) or will show an info icon with a message stating they have the right pieces but in the wrong colours. For simplicity, both the ‘My Pieces’ and ‘Library’ pages feature a home icon for easy navigation to the home page, where users can initiate a new scan, access their inventory, or explore the library. If users choose to initiate a new scan, their current inventory will be erased, and they will start from scratch.

The second flow of the application is for returning users who already have an inventory within the application. Upon opening the application and seeing the splash screen, they will be directly taken to the home page.

6.1.1 Current Frontend Features

To date, we have successfully implemented all user interface screens that the user will encounter within the application and have established smooth navigation across them.

These screens include:

- Splash Screen

1:28



Figure 22: Splashscreen Page

- Instructions Page

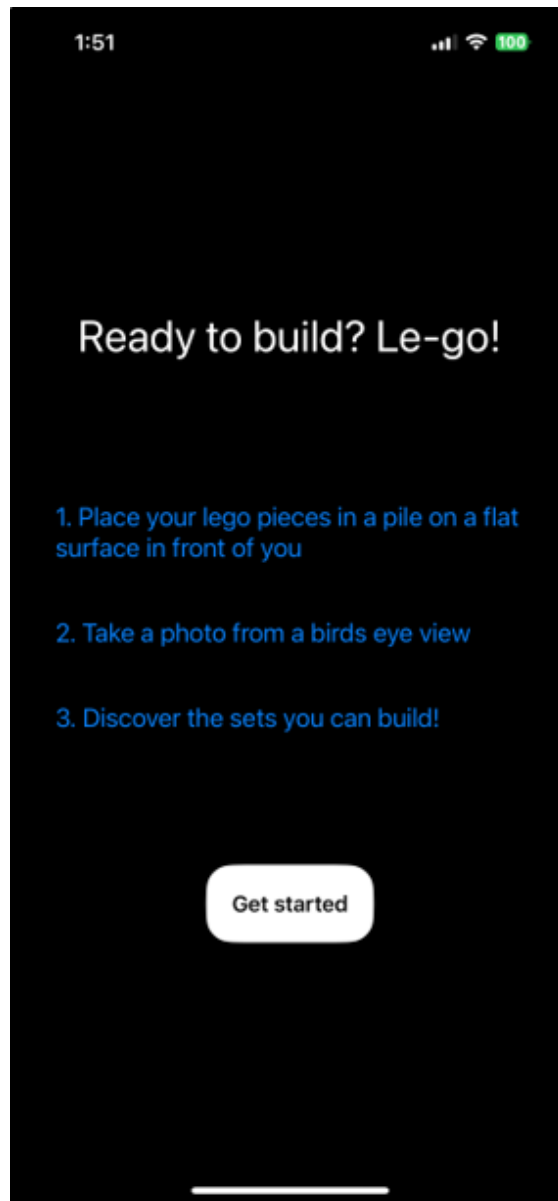


Figure 3: Instructions Screen

- Live Camera

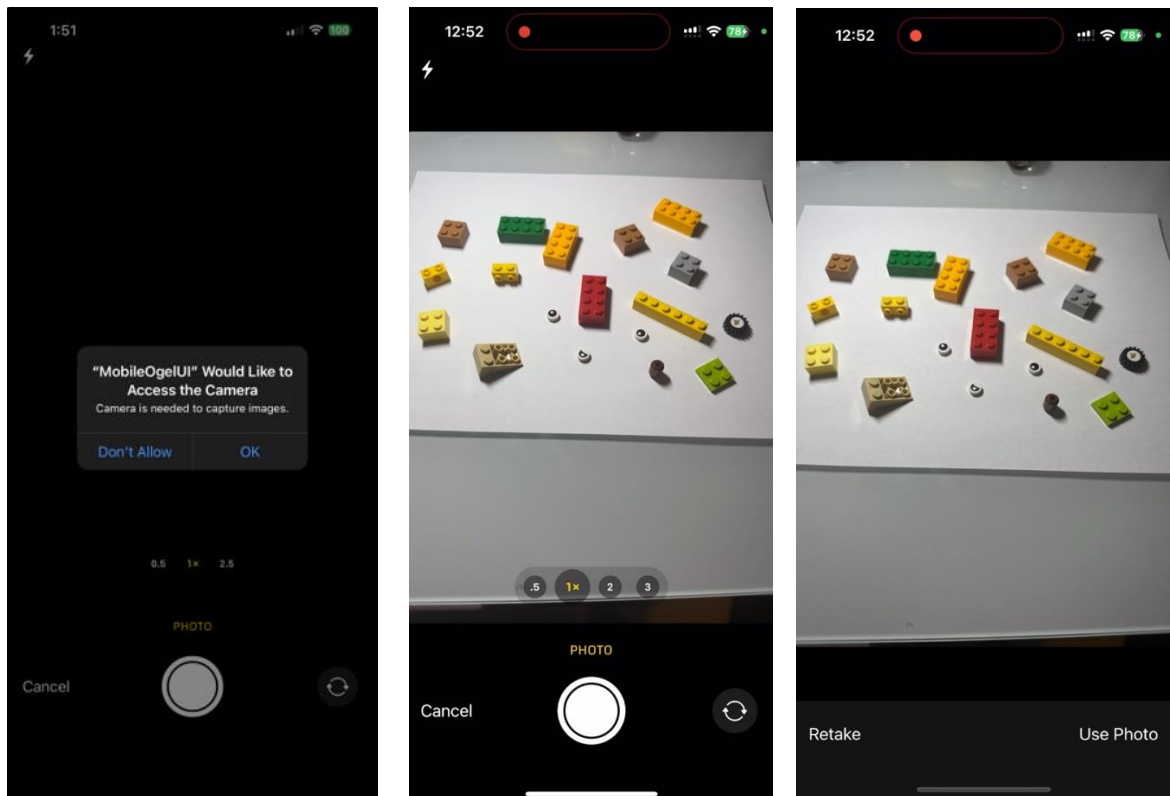


Figure 4: Live Camera View

Note that the live camera does not launch unless the user has granted permission. Once a photo is taken, the user can retake the image or proceed.

- Photo Confirmation Page

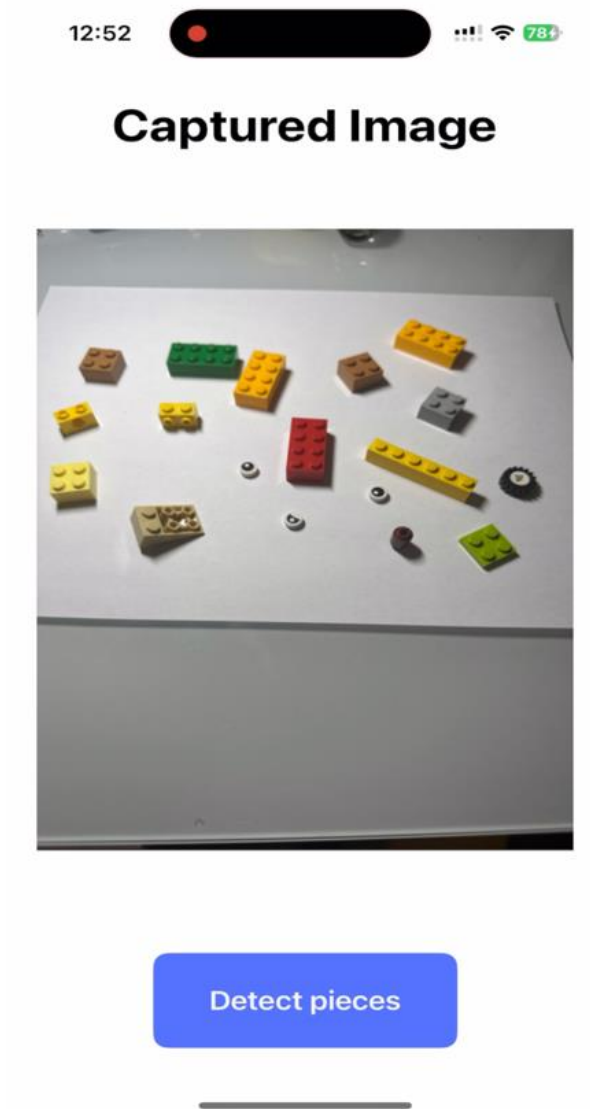


Figure 5: Photo Confirmation Screen

- Scan Summary Page



Figure 66: Scan Summary Page

- Home Page

12:45



Hi!



Scan Lego

My Pieces

My Library

Figure 7: Home Screen

- My Pieces Page

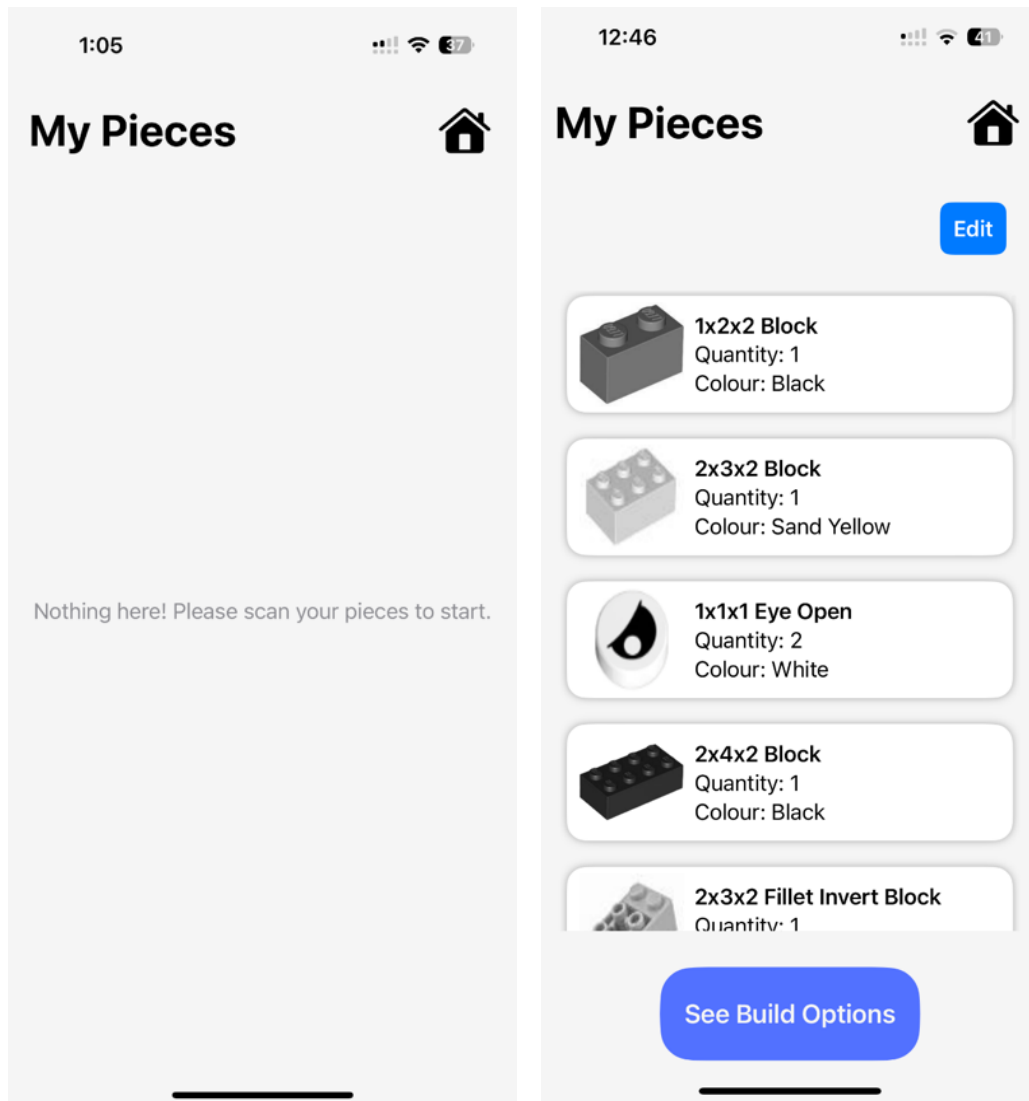


Figure 8: My Pieces Screen

Depending on if the user has scanned or not, there are two different views of the My Pieces page. The image on the right shows the pieces that the model has detected while the image on the left will show nothing if it has not gone through the detection process.

- Edit Pieces Page

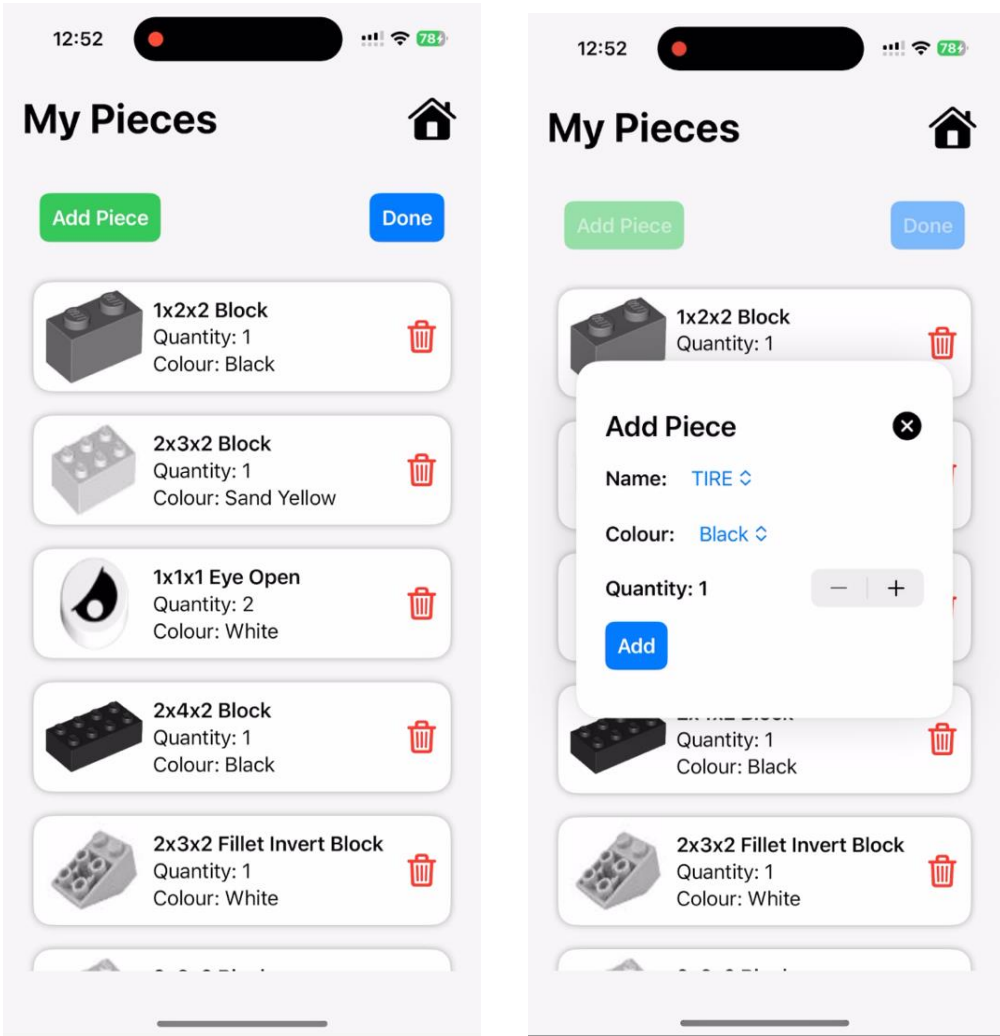


Figure 9: Edit Pieces Screens

- Library Page

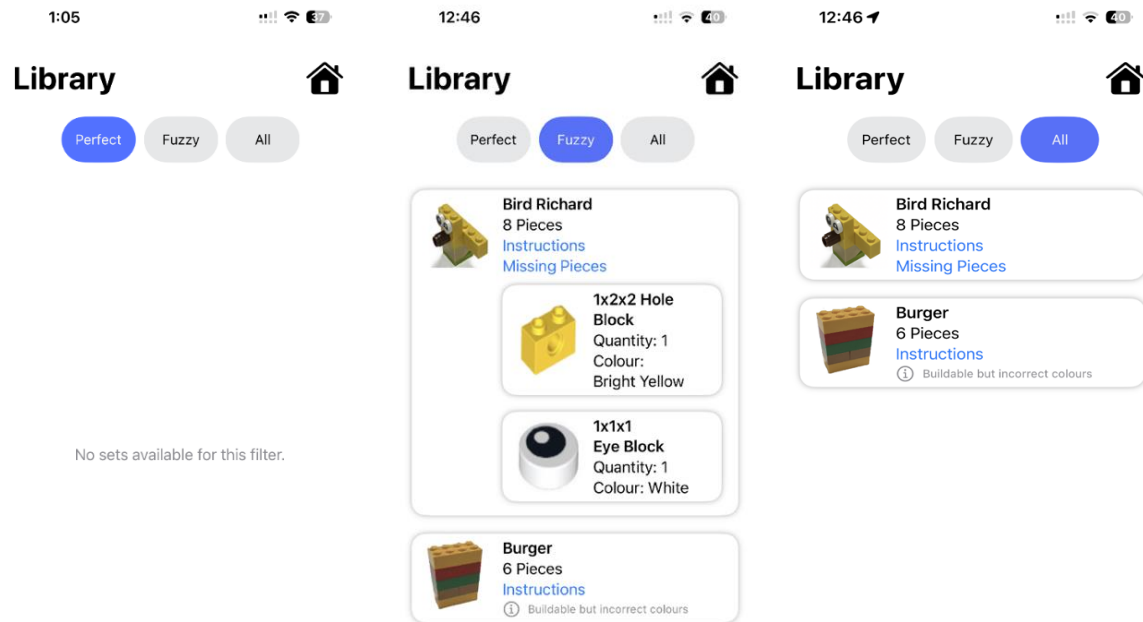


Figure 10: Library Screens

Once again if the user has not gone through the scanning process, the 'Library' page will be empty as seen below.

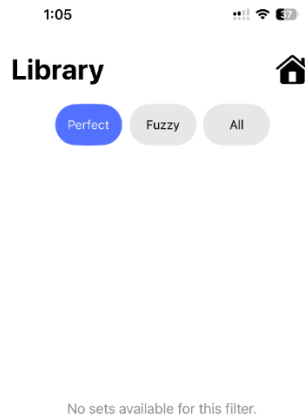


Figure 11: Empty Library Page

We successfully implemented the live camera view by integrating the devices' camera within the application to complete the 'Scan' feature. Users can tap on the 'Scan' button in the application to open the device's camera and capture a photo of their Lego pieces to be sent to the model. It is important to note that if the user is launching the app for the very first time, the application will automatically navigate the user through the scanning process for simplicity.

We also integrated our trained machine learning model within the application to present accurate detections to the user. With the correctly formatted data from the model, we persist the detected Lego pieces to our client-side SQLite database. Along with this, the application will also make a few API calls to our server so it can fetch which Lego sets are compatible with the detected pieces.

With the complete integration of our client-side and server-side databases, the 'My Pieces' and 'Library' pages successfully retrieve and display real data based on what is persisted in their respective databases.

6.1.2 My Pieces Page Display Logic

The My Pieces page offers users the ability to manage their inventory and fix any mistakes the model may have made while detecting pieces. By tapping the 'Edit' button, users can enter edit mode, allowing them to delete detection errors or add new pieces. As seen in Figure 7, while in edit mode, an 'Add Piece' button is revealed along with trash icons next to every piece in the presented inventory list. Clicking a trash icon deletes the corresponding piece from the inventory and updates the local database accordingly. When users click the 'Add Piece' button, a popup appears, prompting the user to input the name, colour, and quantity of the new piece. Selections are made via dropdown menus and steppers to prevent user input errors. Upon confirmation, the new piece will be added to the inventory list and persisted in the local database. In order to navigate to the Library page, the user will have to exit the editing mode by clicking the 'Done' button.

It should be noted that logic has been put in place to update existing pieces if the user has chosen to add a new piece with the same name and colour of a piece that already exists in the inventory list.

6.1.3 Library Page Display Logic

The Library page although simplistic in nature, has a lot of work happening in the background when deciding which sets to display. As mentioned in the beginning of this chapter, the Library page filters between sets, perfect, fuzzy, and all.

Perfect sets are queried by fetching all the sets from the server, specifically their pieces and set ids, and fetching all the pieces stored locally as a Lego piece object array (localPieces). Next, we convert the pieces data for each set into and Lego piece object array (serverPieces). Now that both pieces arrays are of the same format, we compare the two arrays together to ensure all the pieces we have listed in each set exist in the array of pieces we have stored locally/previously scanned (localPieces). For perfect matches, we ensure that the piece dimensions are the same, we have the same quantity, and the same colour to build that particular set. Once we have iterated through all the sets, we store the sets as a Lego set object in a new array (PerfMatches).

For fuzzy matches, since it occurs after perfect matching, we reuse the fetched data from the server and local database by leveraging the pre-established localPieces and serverPieces arrays. The localPieces array is converted into a dictionary of piece: quantity pairs and we iterate through the serverPieces array for each set to ensure all the pieces of that set exist in the dictionary and are of the right quantity or less than what is in the dictionary. The sets that match the criteria get added to a new Lego set object array (FuzzyMatches).

For the All filter of the Library page, we do a simple concatenation of PerfMatches and FuzzyMatches.

Depending on what filter the user clicks on the Library Page, it will either display the PerfMatches array, the FuzzyMatches array, or the All array. To avoid the creation of the

FuzzyMatches and PerfMatches array every time when displaying the Library page, we temporarily store the matching sets locally in our client-side database. This way, we avoid any unnecessary calls to the server, meaning when the user re-launches the application, the lists are prepopulated by simply fetching the sets stored locally. However, if the user decides to scan Lego again, the Lego set table stored locally in the client-side database gets wiped to prepare for the querying of the new pieces scanned.

6.2 Backend

The backend has two sides, a client-side system and a server-side system, that are utilized to create the proper and smooth operation of the application. The following sections will break down the specifics of the backend in the application.

6.2.1 Client-Side System

The client-side is responsible for storing the inventory of scanned Lego pieces and holding the machine learning model for object detection. The client as a result is considered a *fat client* since it will process some application logic embedded into the client-side.

6.2.1.1 Inventory Management

An SQLite database has been integrated into the application to handle the storage and retrieval of the user's Lego piece inventory data. The database schema is designed to reflect the structure of Lego pieces. This includes defining tables such as 'pieces', where each piece is uniquely identified by an auto-generated ID along with attributes like 'imageName', 'pieceName', 'colour' and 'quantity'. Comprehensive functionalities for Create, Read, Update, and Delete (CRUD) operations have been implemented to ensure efficient and robust data management capabilities based on user actions.

It should be noted that when a user chooses to take a new scan, their previously detected pieces are wiped from the database, and the newly detected pieces are inserted. This mechanism ensures that the user's inventory results reflect the most recent scan accurately, maintaining data integrity and providing users with up-to-date information about their Lego pieces.

The SQLite database is integrated into the application utilizing the swift SQLite library. The pieces are stored and accessed by the front-end code running swift methods that will connect to a SQLite file located in the applications file structure and close the connection when access to the database is no longer needed.

6.2.2 Server-Side System

The server-side consists of a database that contains the Lego pieces needed for a set and a link to a PDF copy of the instructions to build the set. The server is responsible for handling the requests from the client asking for instructions. The application will send the server a request for the entire collection stored on it. The application then filters which sets can be built from the scanned pieces. This method of querying or filtering was used due to the timeline for completion of the project and the complexity of the fuzzy matches. The application will store all relevant matches and will disregard the rest of the instruction sets.

6.2.2.1 Assembly Instruction Management

The Assembly Instruction Management system is a non-relational database using MongoDB to store each instruction set as a document. One document equals one Lego set that can be built. The document stores: an id value for the document, an integer value for the Lego set id, which is created by Lego, a string containing the name of the Lego set, an integer value of the total number of pieces that are required to build the set, an array of pieces objects that are required to build the set, and a string that contains the URL where the instructions PDF is stored. The pieces objects have the following structure: an integer for the Lego piece id number, assigned by Lego, a string containing the name/dimensions of the piece based on the machine learning models naming conventions, an integer for the quantity of the pieces, and a string that contains the Lego colour name for the piece, assigned by Lego. Both the document structure and the pieces' object structure align with the structure of the Lego set model for the frontend and the Lego pieces model for the frontend.

6.3 Machine Learning

The ML portion of MobileOgel consists of two main components: brick detection and colour detection. Brick detection identifies the geometry of a Lego brick using the YOLOv5s model trained on a custom dataset. Colour detection is responsible for determining which official Lego colour each individual detected brick is and is achieved using an algorithm the team developed. Colour detection was added later in development, so earlier versions of the model did not describe the colour of pieces.

When an image is passed to the ML module, the model first detects the geometry of the bricks. It then passes the same image along with the location of each detected brick in the image to the colour detection module.

6.3.1 Model Prototypes

Training of the model went through multiple iterations. Iteration 1 was trained on a dataset comprised of digitally rendered Lego pieces and trained for 100 *epochs*, resulting in a model that could not generalize on real pictures of Lego bricks.

Iteration 2 built off the first and used the same dataset from iteration 1, merged with another dataset that included both digitally rendered pieces, and images of real Lego blocks.

Additionally, the technique of transfer learning was employed in iteration 2 (and every iteration after), where the YOLOv5s general purpose image detection *weights* were used as the baseline for training. This improved the accuracy of the model but not enough for a functional demo.

Iteration 3 used a further expanded dataset composed of annotated snapshots of videos our team took of collections of Lego bricks, as well as an expanded list of pieces. To reduce the complexity of the model, the dataset was limited to bricks that existed in the Lego set we used for testing. This improved the model's ability to generalize to images in new environments. However, the updated dataset had many over-represented and under-represented classes, making this model prone to mislabeling pieces as those that were over-represented and often missing under-represented pieces.

The final iteration of the model was used for the demo at the poster fair. The dataset from iteration 3 was edited to include more images of under-represented classes and some images of over-represented classes were removed. While not completely solving the representation problem, this model was considerably accurate and usable within the application.

6.3.2 General Dataset Information

Synthetic and Real Images

The primary problem with the widespread adoption of computer vision systems is the scarcity of annotated images and the high costs associated with acquiring such data [23]. In our case, datasets of real picture of Lego pieces are scarce and usually quite small. To account for this, our dataset contains images taken from the real world and images created through a computer render.

Having a good detection model relies on a dataset that once interpreted has a good detection average, so we can trust the results. Computer rendered models, CAD, or Computer-Aided Design, involves using computer technology to create precise 2D and 3D models of products, buildings, or systems [23]. Rendering techniques simulate how light interacts with surfaces to produce realistic visual representations. These synthetic images can help provide full control over various factors such as lighting, camera angle, and object placement. This control is beneficial for many testing scenarios since it provides the algorithm with similar images to improve the model stability. Creating synthetic images that are very realistic can be challenging which is why it is important to have real world images in the dataset as well [23]. With a real-world dataset, the model will be more prepared to handle more complex scenarios with the diversity of the real world.

We can see from these graphs that a model trained with both types of images performs better together than individually.

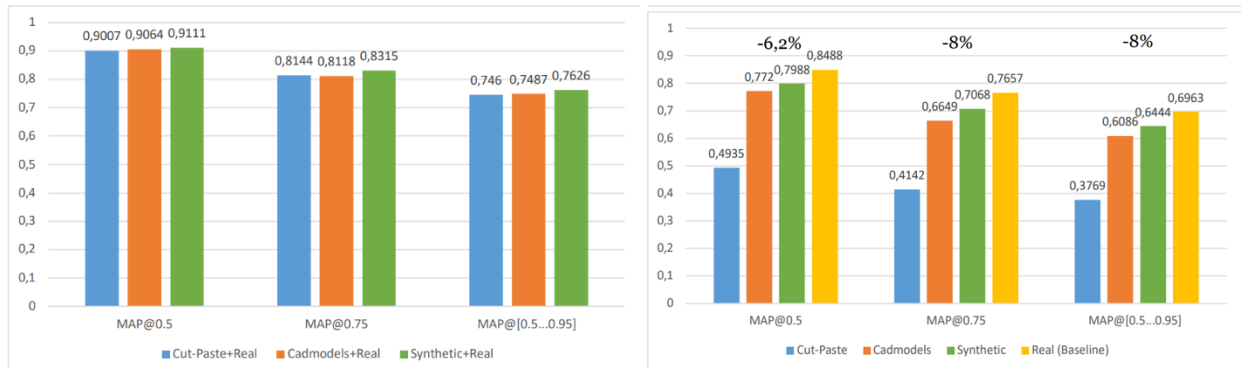


Figure 10: Comparison of Techniques [5]

For this project we need a good mix of images of individual pieces and pieces in groups, to simulate a real environment. Due to these constraints, there are very few freely available datasets at our disposal. In addition to this, creating a large dataset with real images is time consuming, difficult, and as shown by Figure 3.3.1.1, is not entirely worth it. For these reasons, we have opted to use synthetic and real images in our dataset and will continue to append to it with new images we take.

6.3.3 Current State of the Model

The current state of the model is functional in detecting a subset of Lego pieces that we deemed to be the most important. We have worked on balancing our dataset to have more well-represented classes. Our class representation can be seen in the figure below.

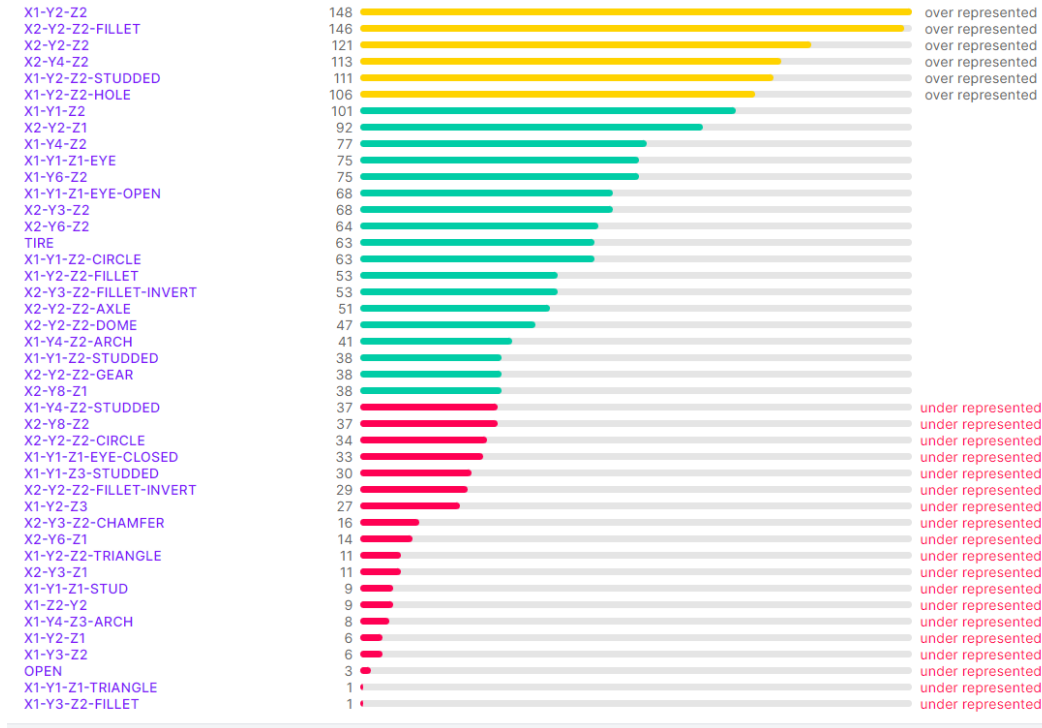


Figure 12: Dataset Health

Our model has many properly represented classes in the green allowing for it to be able to better classify the images given to it. Where it lacks is in the over/under-represented classes, those classes are what currently give our model uncertainty in its decision-making.

One of the performance metrics we used for determining the performance of our model was the mean average precision (mAP). With the discrepancy in our dataset health, we were still able to achieve a good mAP. Our model's mAP was 0.845 which means most of the predicted positives are likely to be true positives. This was important since we did not want the user to have to edit the output from the model often.

For our well-represented classes, the model was able to also produce a good precision-recall (P-R) curve for our standards. The figure below shows the P-R curve of our model:

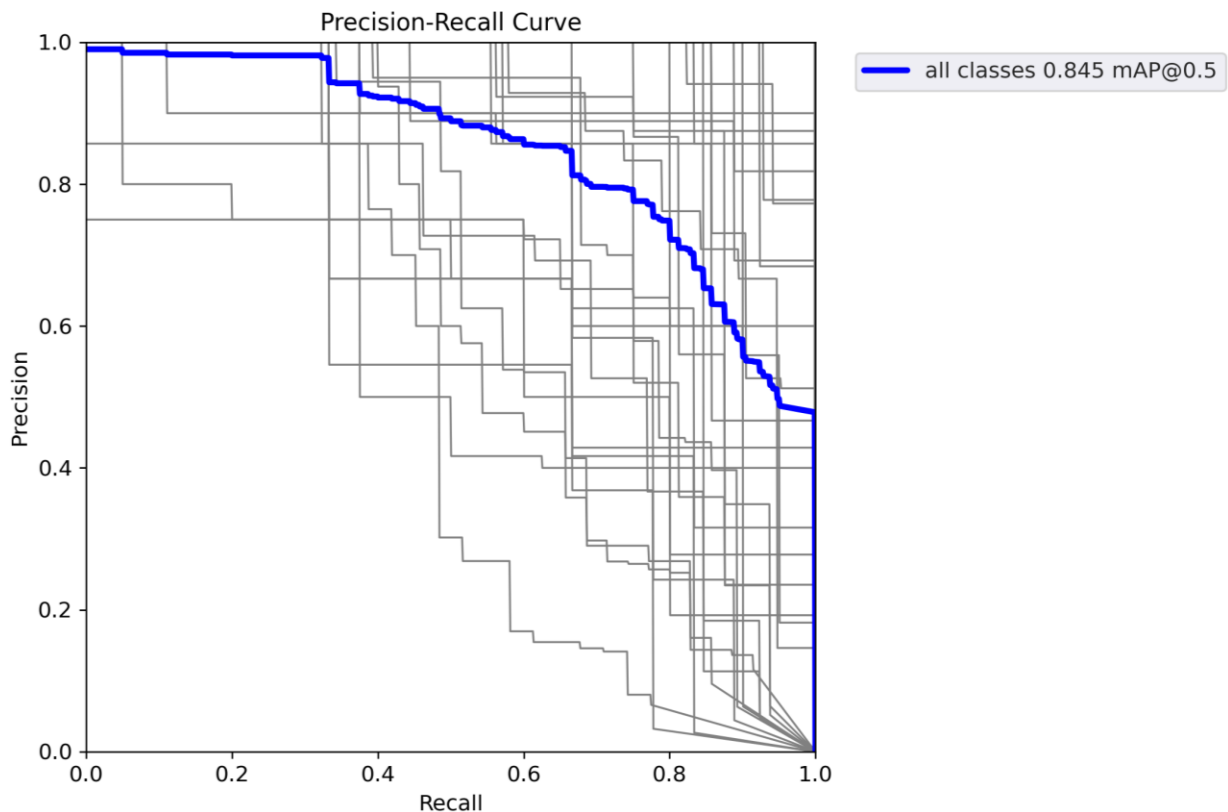


Figure 13: Precision-Recall Curve

The grey lines indicate the curve for each of our classes. Our well-represented classes have a high area under the curve (AUC). The higher area indicates the overall performance of our model across all possible thresholds. The AUCs closer to one means the model can perform well for those classes.

Although our model's metrics were high, testing practically we did not always see the best results when it came to detection. The model's performance depended on many conditions we hoped to remove with the training like angle, lighting, and distance. To achieve the best results when deploying the model, the user would have to make sure the picture was taken at certain angles to obtain the best results.

Even through these shortcomings and needed workarounds, our model was able to meet our self-set standards for performance in the time frame. We are satisfied with how our model performs in detecting Lego.

6.3.4 General Model Information

Classes

Classes refer to the distinct categories that a machine learning model classifies input into. In the context of computer vision, classes refer to the objects the model has been trained to identify and are usually provided in a '.yaml' file in addition to a label on each image in the dataset denoting which class it belongs to. In the case of Mobile Ogel, these classes represent the different types of Lego pieces that exist and are named in the following convention:

$X\{i\}-Y\{j\}-Z\{k\}-GEOMETRY$

- X – The width dimension of the Lego block
- Y – The length dimension of the Lego block
- Z – The height dimension of the Lego block
- i, j, k – Represent the respect width, length, or height of the Lego block
- *Geometry* – Represents the unique geometry of a particular block (i.e., Chamfer, Fillet, etc)

Dataset and Training Specifications

The dataset used for iteration 4 consists of 2480 images after augmentation. The dataset is split into 3 categories as is standard in training computer visions models:

- Training (72%): The model trains on these images only, 1 epoch denotes one pass through each of the training images.
- Validation (20%): After each epoch, the model is run against this dataset to measure its precision and accuracy throughout the training of the model.
- Test (8%): Used to judge the model's performance after training is completed. Final metrics are calculated based on the classification accuracy and precision on this dataset with a set of weights chosen to test on.

Several augmentations were applied to images in the dataset to both increase help with under-representation of certain classes, and help the model generalize in unfamiliar settings.

- Rotation: A 90-degree rotation was applied was applied at random to some images to help it deal with different camera orientations.
- Shear: A -10-to-+10-degree shear was applied at random to some images to help the model understand different perspectives users might take pictures in.
- Grayscale: Applied to a large portion of images to help the model make inferences on shape without taking colour into account.
- Exposure: A -10% to +10% exposure change was applied at random to some images to help the model infer accurately in different lighting conditions.

Upon training, images are resized to 640x640 pixels. This is the default sizing recommended for training a YOLOv5 model. The square aspect ratio simplifies the anchoring process of the model and in a more general sense, resizing the images provides standardization in our dataset. By enforcing a standard size, we can eliminate image size as a skewing factor in the accuracy of the model. The caveat of this is that images used in inference, like those taken by users in the application, must also be resized. There are important considerations to be made in terms of the

size of the images, smaller sizes mean faster inference, but lower accuracy compared to larger images. As it stands right now, the model's performance is adequate at the default size.

6.3.5 Training Techniques

The goal of training a model is to get it to a point where the weights used in inference maximize the accuracy and precision of the model. The main technique used for this in iteration 2 of the model was "Transfer Learning".

Weights

In the context of computer vision and machine learning, "weights" refer to the parameters of a model. The model learns to make predictions by adjusting the numerical values of its internal parameters, which are called weights [2]. Tuning parameters is what allows models to begin to recognize patterns.

YOLOv5 generates weights as ".pt" files. At the end of training, 2 weights files are generated, "best.pt" and "last.pt", representing the weights that performed the best against the validation set, and the latest weights generated in the last epoch of training. It is important to note that often "last.pt" and "best.pt" are not the same, and either one can outperform the other in real world applications [38].

Transfer Learning

"Transfer Learning" is a machine learning technique where a model developed for a particular task is reused as the starting point or baseline for a model on a second task. This is done by taking the generated weights of the first model and using them as the baseline to train the second. This is particularly useful when the use case of the second model has less data available for training. In addition, "Transfer Learning" is an effective method to reduce computational resources to train a capable model. Providing a good and relevant baseline for the model cuts out the initial stages of training the model where parameters are initially established [38].

In the context of Mobile Ogel, most freely available datasets were digitally generated and not up to the standard we envisioned. Also, it may be infeasible to create our own dataset large enough to effectively train a model from scratch that can recognize Lego effectively, especially in environments different than those in the training dataset. Further, our own computational resources are limited and by using a baseline, we can save on computation costs. We employed transfer learning in our model by using "general purpose" image detection weights that are already able to classify and detect basic shapes. These weights were provided by Ultralytics, the organization responsible for creating YOLOv5.

Platform

Currently, to train the model we use the Google Colaboratory (Colab) platform. It is an online collaborative python notebook, allowing documentation, visualization, and code to live in one spot. It also allows for steps of the training process to be modularized and run independently, which helps make training iteration and debugging more efficient. We purchased the Colab Pro subscription for \$13.99/month which gives us access to stronger GPUs and more computation time.

The notebook used for training and validation can be viewed here:

<https://colab.research.google.com/drive/1utHqZuBxaR6mZvWx9QpwmXNcGBokrect?usp=sharing>

6.3.6 Metrics

There are 3 main values used to judge or assess a computer vision model [37].

- Precision: The ratio of true positive predictions to the total predicted positives. It measures the accuracy of the positive predictions made by the model. High precision indicates that when the model predicts a positive class, it is likely to be correct.

$$\text{Precision} = \frac{\text{TruePositives}}{\text{TruePositives} + \text{FalsePositives}} [40]$$

- Recall: The ratio of true positive predictions to the total actual positives. It measures the ability of the model to capture all positive instances. A high recall indicates that the model is effective at identifying all positive instances.

$$\text{Recall} = \frac{\text{TruePositives}}{\text{TruePositives} + \text{FalseNegatives}} [40]$$

- Accuracy: The ratio of correctly predicted instances (both true positives and true negatives) to the total instances. It provides a general measure of overall correctness. Good metric for balanced dataset, but imbalanced sets with more positive than negative test images or vice versa can be skewed.

$$\text{Accuracy} = \frac{\text{TruePositives} + \text{TrueNegatives}}{\text{All Inferences}} [40]$$

Independently, these metrics do not do a good job of giving a clear picture of the model's performance. To deal with this, these values are often combined to provide a better understanding of the model's performance and pitfalls.

F1 Score

Combines precision and recall into a single metric. It is particularly useful to measure performance with an imbalanced dataset or an uneven class distribution. In the context of MobileOgel, a much larger presence of a certain Lego block over another in the training set.

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} [40]$$

F1 has an upper limit of 1, the higher the value, the higher the precision of the model. An F1-Confidence curve can help determine at which confidence level the model should classify an inference as “positive”. Mapping the F1 at different confidences gives an idea of how confident vs accurate the model is.

Mean Average Precision (mAP)

This metric is the average of the “Average Precision (AP)” across all classes. AP maps precision against recall, seeing the precision of inference at different recall levels.

$$AP = \int_0^1 p(r) dr [37]$$

$p(r)$ is the precision at a given recall r

$$mAP = \frac{1}{n} \sum_0^n AP_n$$

Where n represents the number of classes trained on [40]

The closer mAP is to 1, the better the performance of the model is considered. A $mAP = 1$ means that the model recalls every class perfectly, with perfect precision.

Confidence

The model itself outputs a value upon inference that denotes how confident it is in the classification it made, this value ranges from $[0,1]$, the higher the value, the more confident the model is in the prediction.

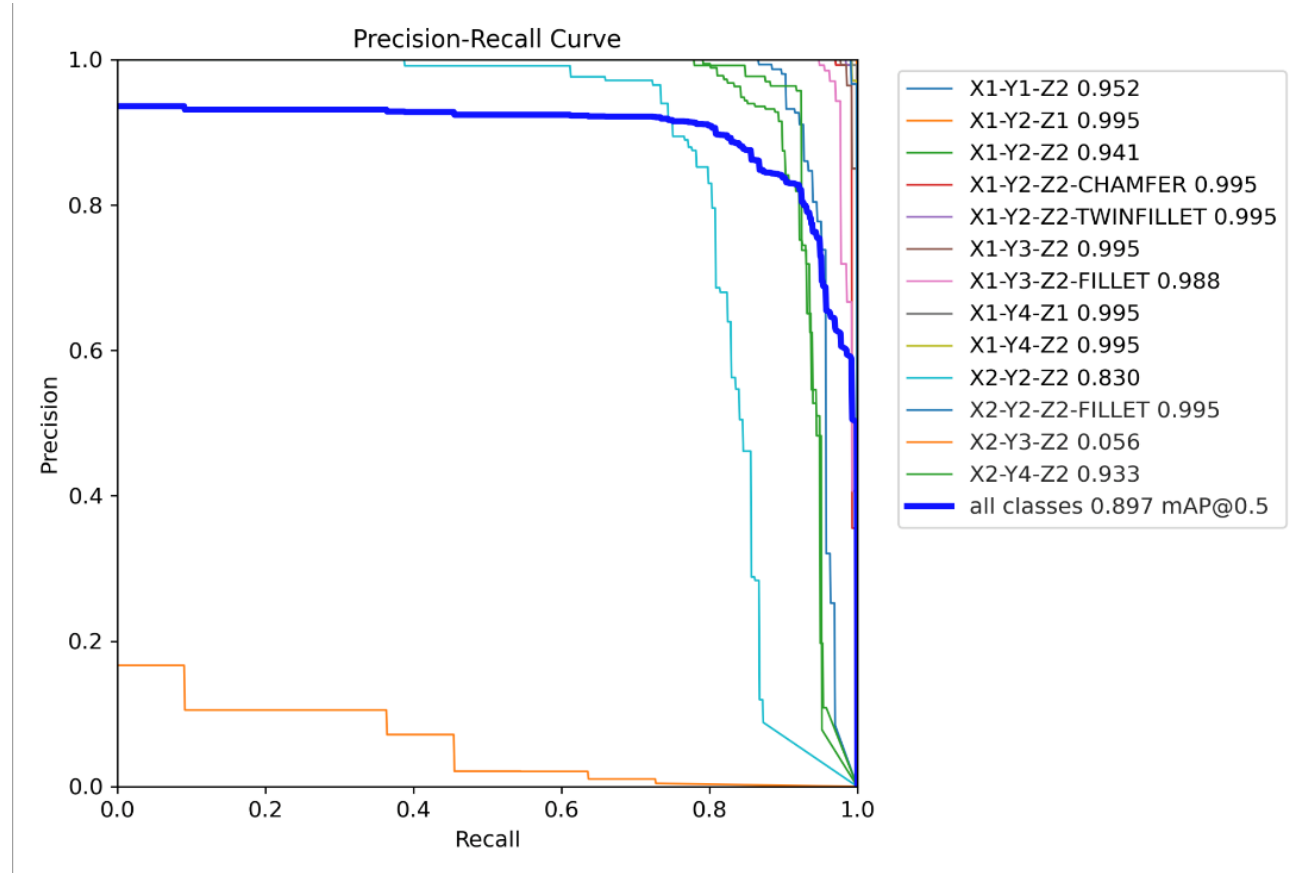


Figure 14: Precision-Recall Curve for Iteration 2 of Model, used to Calculate mAP

F1

The F1-Confidence curve indicates that there is a good balance between precision and recall in the model. The model can both correctly find Lego pieces where they exist and correctly classify them. The steep drop-off in the figure at a confidence of 1 is expected as the model will almost never detect Lego within the images with 100% confidence. The poor F1 score for “X2-Y3-Z2” is further proof that it is under-represented somehow within the dataset and will need to be explored.

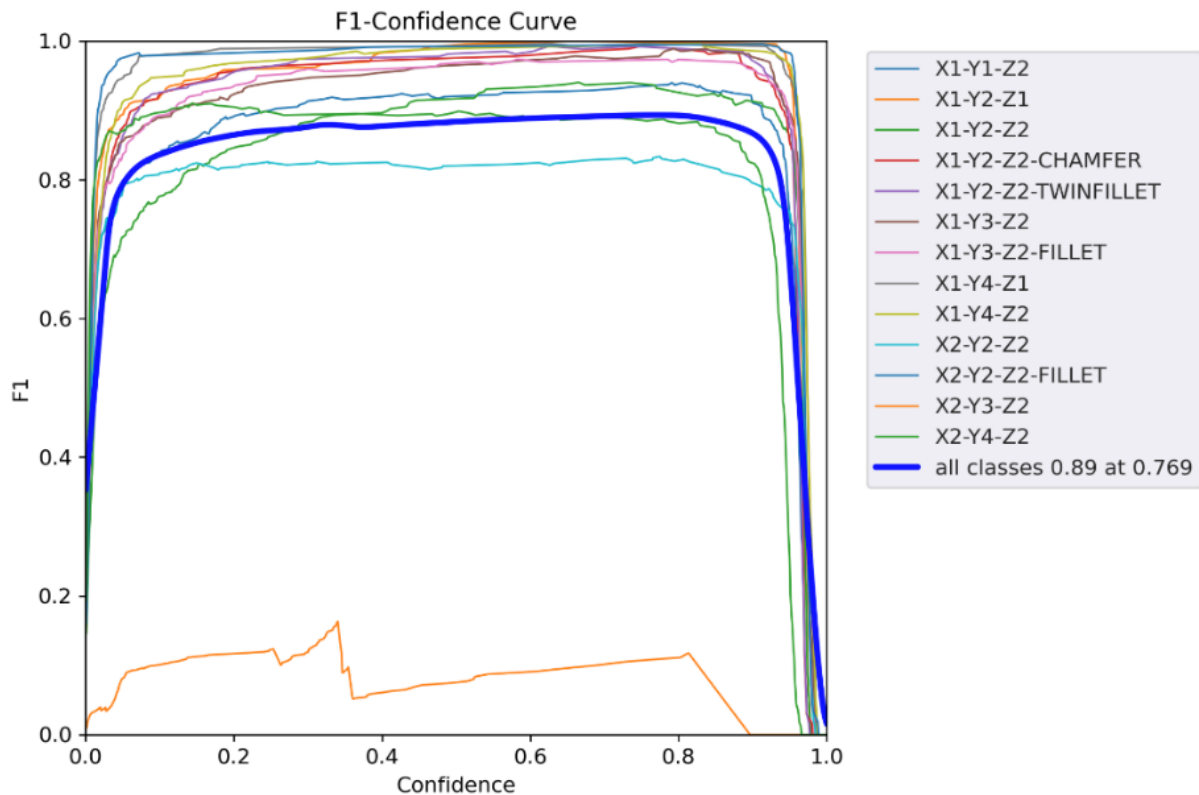


Figure 15: F1-Confidence Curve for Iteration 2 of Model

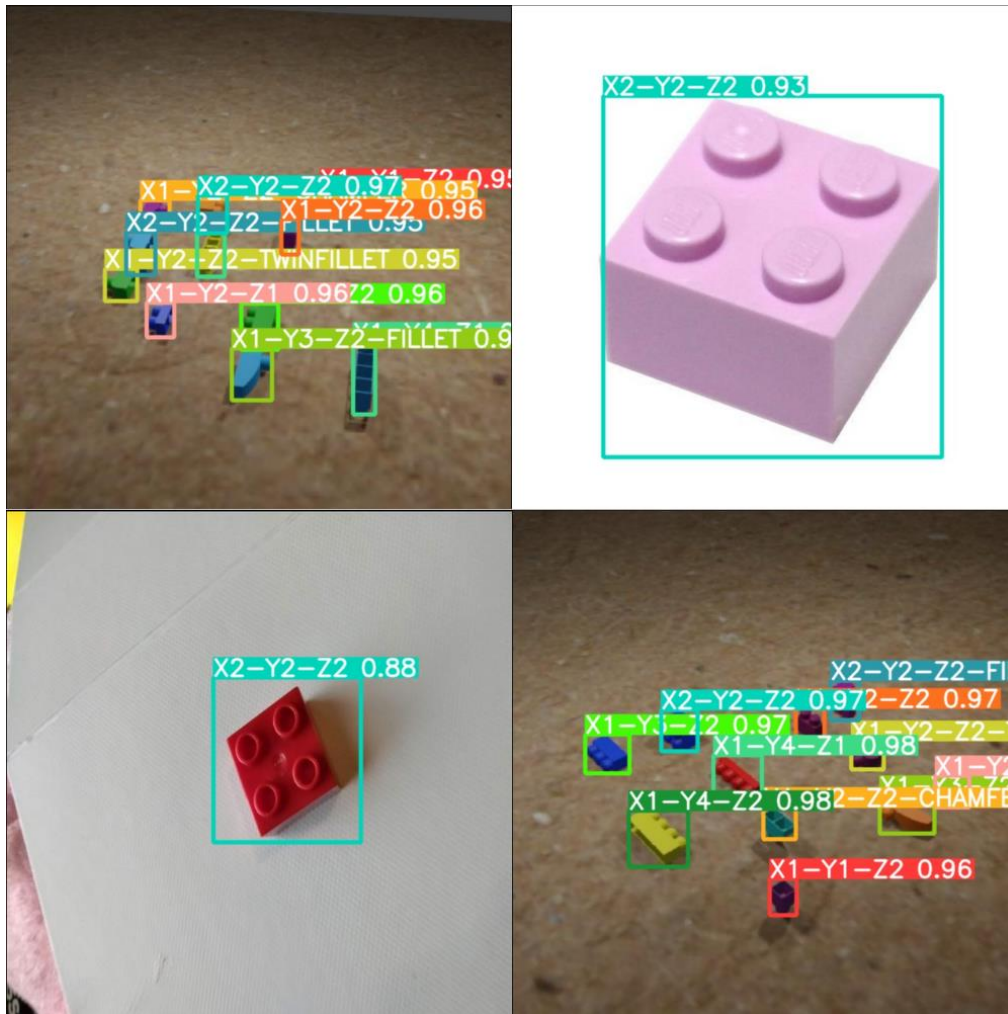


Figure 16: A collection of successful inferences on both real and digitally rendered pieces, confidences can be seen as the right most value on the label

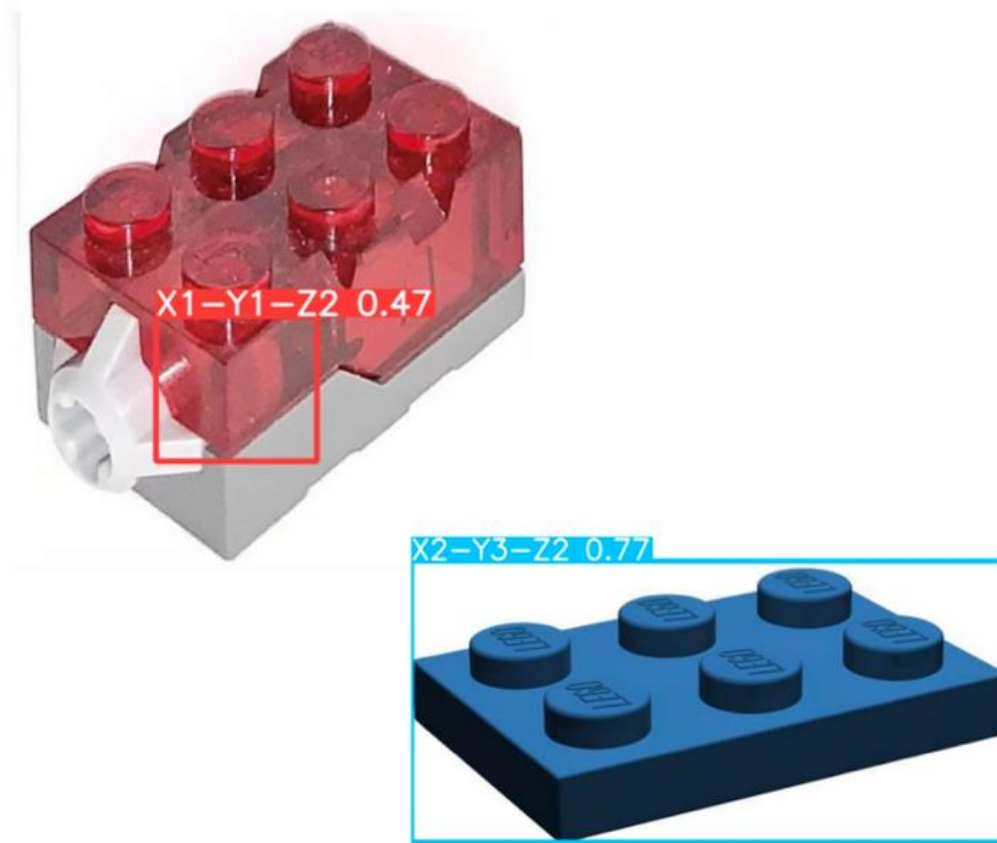


Figure 17: Examples of successful detection but incorrect classification using our model

6.3.7 Translation of the Model

With the model development complete, to integrate our model with the IOS application, we must translate the model into a format IOS understands, namely into a CoreML format. To translate our model from YOLOv5 into CoreML, it is a three-step process defined below:

1. Export the model from YOLOv5 into CoreML using the export method defined in the YOLOv5 package. This outputs a grid of bounding box predictions, their classes, and confidence score. The model is in a format that is not human readable.
2. Add a decode layer. The decode layer is responsible for transforming predictions into actual bounding image coordinates.
3. Add a Non-Maximum Suppression (NMS) layer. This is a post processing technique that selects the bounding box with the highest confidence score, allowing us to accurately pinpoint a specific Lego piece.

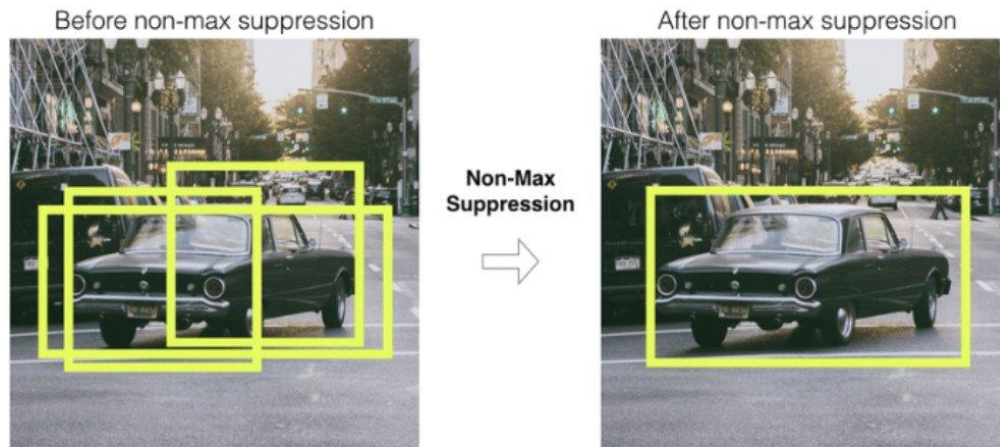


Figure 18: NMS Visual Representation

Looking at Figure 18 above, the image on the right indicates what the model will output after step 2. At this point, the model can detect the car but has not quite figured out what bounding box (yellow box) best represents the car. After adding the NMS layer in Step 3, the model will select the bounding box that best represents the car based on the highest confidence score as seen in the image on the right.

6.3.8 Colour Detection

Once pieces are detected in the image, their location on the image is passed to the colour detection module. The goal of the colour detection algorithm is to determine the official Lego colour of each detected Lego brick.

The Algorithm

Each detected brick is analyzed by sampling pixels within its bounding box and determining which official colour it is closest to. To achieve this, a list of official colours and their RGB values was compiled through sources like BrickSet.com.

The RGB value of a pixel on a detected piece is then compared to each official colour in the list by treating both RGB values as 3-dimensional vectors and measuring the Euclidean distance between them. The official colour with the smallest Euclidean distance to the pixel is considered to be the colour of the pixel.

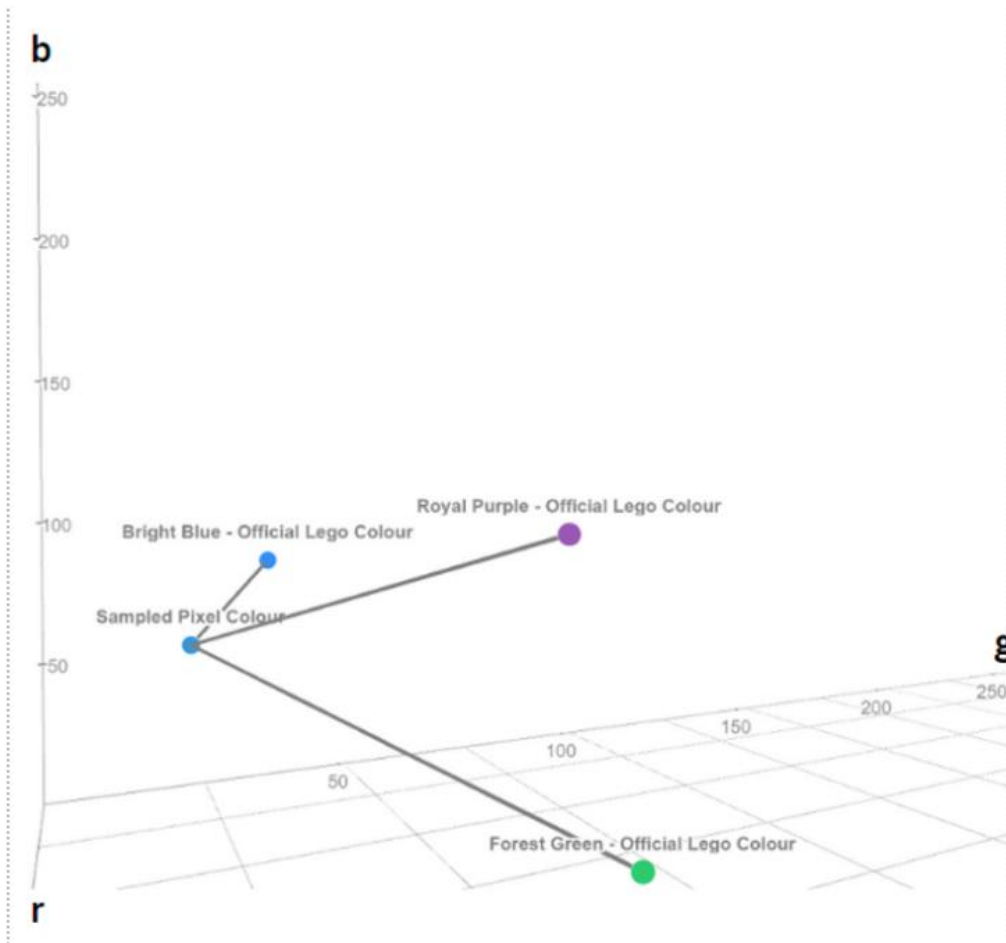


Figure 19: Visualization of the Euclidean distance between a sampled colour and official Lego colours

Determining How Many Pixels to Sample

Determining the colour accurately with one pixel can be challenging as any given detected brick could be obfuscated by another brick, have glare on it, or be affected by any other lighting. However, sampling every pixel within the bounding box of the detected brick is too computationally intensive.

To find a suitable middle ground, a statistically significant number of pixels is determined using a statistical power analysis. This allows us to estimate the appropriate number of pixels to sample given the area of the bounding box, a margin of error we are comfortable with (5%), and an effect size determined experimentally through trial and error by our group [52].

The determined number of pixels are randomly sampled from within the bounding box of a brick. Each pixel gets an associated official Lego colour and the official colour seen the most in the sample is said to be the colour of the entire brick.

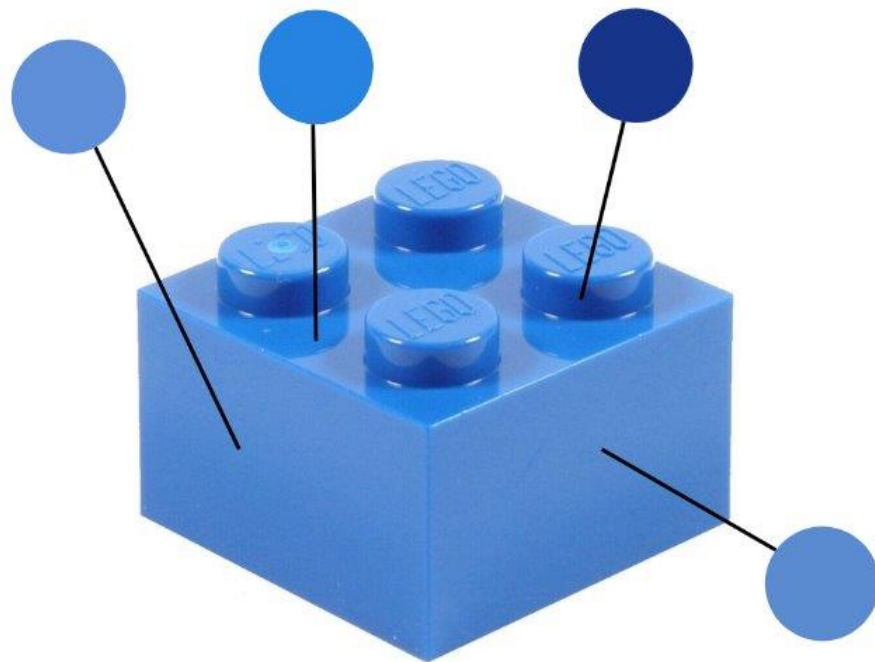


Figure 20: Visualization of different areas of a Lego brick showing different colours.

Chapter 7 Testing

Throughout our application's development, we have conducted rigorous testing through manual tests, unit tests, and user testing. Conducting manual tests and unit tests throughout the development process helped us catch bugs early and better understand what user experience we wanted to achieve.

7.1 Manual Testing

Throughout the development of our application, all team members conducted manual testing to ensure pages and buttons worked as expected when the application was deployed to phones for testing. This helped us better understand the flow of the application along with what we wanted the navigation experience to look like. Every feature that was developed was tested manually on a device to ensure the application was functioning as expected. For example, when we had written queries to query our server, our unit tests were passing but the connection failed on the device. Having caught this issue, we were able to update our server connection string so the application could connect once deployed on the device. Given our application is meant to be deployed on a device, manual testing had to be conducted to ensure all features are functional “in production”.

7.2 Unit Testing

Throughout our application's development, we wrote unit tests to conduct regression testing on the application's overall functionality. We leveraged XCTest which is a test framework to write unit tests for XCode projects. Before local code could be merged into our dev branch, it had to pass a series of unit tests to ensure no core functionality broke. This was an automated process to ensure incoming code was not creating bugs or causing regressions. Unit tests were added and updated as more features were added to the application. Some things like taking an image (camera functionality) cannot be fully tested through XCTest but all functions and connections were covered through unit testing. Additionally, some tests involved the use of mock data to isolate the code being tested from its dependencies, such as the local database. For example, to test the perfect and fuzzy match logic, mock data was used to ensure tests focused solely on the unit's behavior. By employing unit tests, our team was more comfortable approving PRs and taking risks as we knew the code being merged would be functional.

7.3 User Testing

Once our application was in a state where it was useable and could generate results, we conducted a pilot study with fifteen participants. Participants were recruited by word of mouth and mostly friends and family. Each participant was tasked with using the application to scan Lego pieces and provide feedback. Participants employed the “think aloud” approach, where they had to voice their thoughts as they were completing their task. This helped us better understand user frustrations and the things they liked as they used our application. For example, some users pointed out that our buttons were not salient enough on the home page, so we made

them bigger. Also, some users wanted more control and freedom over the navigation experience so they can return to the home screen or exit at any time. This led to the addition of the home icon on all UI pages and implementation of the cancel button when the user opens the camera. Additionally, one user made a comment about how they liked that our colour scheme was accessibility friendly. User accessibility was important to us, so we made sure our colour scheme was accessible and put all colours through a contrast checker and ensured it was compatible with W3C accessibility guidelines. Compiling all our feedback, all users enjoyed the overall experience of using our application and could see themselves using the application when playing with Lego.

7.4 Requirements Testing

Through our application development cycle most of the systems' more general functional and non-functional requirements were measured and tested. This section will go over how we specifically tested the most important requirements of this system.

To ensure compliance with functional requirements 1, 4, 6, and 7, and non-functional requirements 2 and 4, manual testing methodologies were used as mentioned in section 7.1. The measurements outlined for each of these requirements were rigorously tested using various strategies such as conducting tests on various iPhone devices to account for different screen sizes and camera functionality.

Non-functional requirement 7 was tested in the same user pilot study described in section 7.3. Error messages and handling were assessed by simulating error conditions, observing the users' reactions, and asking them to describe what was happening in the system. The errors tested included placeholder images when pieces in certain colours could not be retrieved, when the model failed to detect pieces, and when the application could not connect to the database.

Chapter 8 Conclusion and Reflections

Over our eight-month project, we have learned a lot. Our project aimed to tackle the challenges of surplus Lego pieces and the environmental impact posed by excess pieces not being used. Our application utilized computer vision to categorize and inventory unused Lego pieces and provide users with build instructions for various Lego sets given their existing pieces.

Our solution involved developing a mobile application leveraging machine learning and computer vision to accurately recognize various Lego pieces. The application's functionality includes analyzing the identified pieces and providing build instructions for different Lego sets the user can create.

8.1 Conclusions

Overall, we were able to develop an iOS application using Swift from scratch that could effectively query/communicate with both our databases. Along with this, we learned how to manipulate data and the importance of having design patterns in place. By leveraging MVVM, we ensured that our codebase remained organized and scalable as we introduced new features and functionality. This approach facilitated code management and enhanced collaboration among team members, enabling us to maintain a cohesive development workflow.

Throughout the project, we encountered challenges posed by updates to Apple's frameworks, which required us to adapt our codebase accordingly. Despite these obstacles, our team demonstrated resilience and resourcefulness by effectively interpreting vague documentation and seeking guidance from online forums. This experience not only strengthened our problem-solving skills but also highlighted the importance of staying updated with evolving technology trends.

In the machine learning aspect, we achieved significant milestones in developing a model capable of accurately identifying a large subset of Lego pieces we had available. By employing advanced techniques in dataset construction and manipulation, we optimized the model's performance and reduced class imbalances and over-representation. This iterative process involved continuous refinement of our dataset and training to achieve an accuracy of 85%. The combination of our efforts resulted in an application that seamlessly integrates Lego piece recognition, inventory management, and personalized set recommendations for users. Moving forward, we are committed to refining our application further and preparing it for release on the App Store. We look forward to sharing our product with a wider audience and continuing to evolve and improve it based on user feedback and emerging trends in mobile development.

8.2 Reflections

8.2.1 Alternative Solutions

Looking back on our project, there are a few things that could have been done differently. Initially, we decided to develop the application in XCode because most of our group had interest and prior experience using it to develop iOS applications. One downside to this decision was that there would be no support for android users, and this was a decision we were comfortable making as the demographic we were targeting (younger kids – teens) tend to have Apple devices. One thing we did not account for was throughout the development process, those who were doing application development on Windows laptops were unable to test the code because of incompatibility with different operating systems. If we were to start again, we would have used React Native as all team members could actively participate in the development and testing process. Additionally, applications developed using React Native can be deployed on both iOS and Android devices. From the poster fair, many students with Android devices were disappointed that there was no support yet available for their device because they really liked the application.

Another thing that we could have changed was putting more of an emphasis on ensuring our model had a balanced dataset. Although we started labelling samples early, it was a very tedious process. In the beginning, we not only leveraged existing datasets but also took our own images to cover some pieces that were missing from existing sets. We did not learn till late in the process of tools like Roboflow that we leverage to monitor our model, see which classes are underrepresented/overrepresented, and quickly label images.

Furthermore, although we achieved a decent model in the end, instead of starting from scratch to create a model, we could have tried leveraging a pre-trained model or baseline model to work off of. This could have saved us several rounds of training as pre-trained models have already learned meaningful features and patterns from larger datasets that can be applied to our use case of detecting Lego pieces. Knowing this, we could have applied transfer learning to fine-tune the existing model to our specific case.

8.2.2 Project Extension Recommendations

Our project as it stands can be further extended to include more Lego sets and leverage existing Lego set databases. We can create APIs to interface with these rather than maintaining our own server-side database to store possible Lego set designs. If we were to extend the amount of Lego sets stored in the database, we would want to shift set matching logic like “fuzzy match” processing to be completed on the server side compared to the client side to reduce the amount of data transferred from the database to the client.

Additionally, we believe this project can be modified to solve a common problem of setting up a grocery list and meal planning. We believe that the model and application can be changed to scan the contents of a fridge, identify all the items in it, and report different meals the user can make. The application would be able to store all the items in the user’s fridge and give them meals they can make with their current inventory as well as things they can make if they buy one or two more items. Alternatively, it can also be used to help with interior decorating. The model can be altered to provide furniture layout designs based on existing furniture users have.

8.2.3 Project Positives

As we reflect on our project it is truly remarkable to see all the progress that has been made transforming our vision for the MobileOgel application. Achieving our main goals was a testament to our commitment to making this project a reality and helping people organize and use all their Legos.

Firstly, our efforts to create an intuitive and efficient iOS application were met with overwhelming positive feedback from users. By integrating our interactive scanning and user interface together we left users with a positive experience after using our application. During the poster fair, several people who used the application and heard our pitch expressed how this would reignite their interest in Lego and even asked us to upload MobileOgel to the app store. As we look to the future, our non-focused goals have revealed promising prospects. We believe that despite not completing the entire project's scope, we have accomplished the goals we found most important. We completed the design of our application and provided a great user experience, functionality and aesthetics. Most of the next steps are not heavily engineering and design focused and mostly relate to maturing the application rather than completing its core functionality. As a group, we are happy that we were able to bring it to that point and are confident leaving these next steps for the future.

From an engineering perspective, the development of MobileOgel has had a positive impact on our skills as software developers. We have all expanded our skills related to ML and iOS application development but most importantly our engineering design skills. Throughout the course of this project, we have seen an original idea of ours come to fruition in the form of a fully functional application.

In summary, MobileOgel stands as a testament of the power of merging technology with Lego. Through our iOS application, we have opened a new window for users to interact with their old Lego. As this project continues to expand the possibilities for MobileOgel are endless, promising to inspire and facilitate the creativity of Lego fans around the world.

References

- [1] J. Adamczyk, “Make knn 300 times faster than scikit-learn’s in 20 lines!,” Medium, <https://towardsdatascience.com/make-knn-300-times-faster-than-scikit-learns-in-20-lines-5e29d74e76bb> (accessed Apr. 10, 2024).
- [2] R. Alake, “The Future of Computer Vision,” NVIDIA Technical Blog, <https://developer.nvidia.com/blog/the-future-of-computer-vision/#:~:text=Computer%20vision%20is%20a%20rapidly,to%20them%20in%20real%20time> (accessed Sep. 20, 2023).
- [3] “What is service-oriented architecture?,” Amazon, <https://aws.amazon.com/what-is/service-oriented-architecture/#:~:text=you%20implement%20microservices%3F-.What%20is%20service%2Doriented%20architecture%3F,other%20across%20platforms%20and%20languages>. (accessed Sep. 22, 2023).
- [4] R. Ambadipudi, “Council post: How machine learning will transform your industry,” Forbes, <https://www.forbes.com/sites/forbestechcouncil/2023/02/27/how-machine-learning-will-transform-your-industry/?sh=2c3966991a3b> (accessed Oct. 9, 2023).
- [5] C. S Arcidiancono “An empirical study on synthetic image generation techniques for object detectors”, <https://www.diva-portal.org/smash/get/diva2:1251485/FULLTEXT01.pdf> (accessed Dec 6, 2023).
- [6] Atlassian, “Jira Scrum Boards,” Atlassian, <https://www.atlassian.com/software/jira/features/scrum-boards> (accessed Dec. 7, 2023).
- [7] Atlassian, “What is Scrum? [+ how to start],” Atlassian, <https://www.atlassian.com/agile/scrum> (accessed Dec. 1, 2023).
- [8] “What is Agile?,” Atlassian, <https://www.atlassian.com/agile> (accessed Oct. 20, 2023).
- [9] “What is an event-driven architecture? - aws.amazon.com,” AWS, <https://aws.amazon.com/event-driven-architecture/> (accessed Oct. 25, 2023).
- [10] “What’s the Difference Between SDK and API? - aws.amazon.com,” AWS, <https://aws.amazon.com/compare/the-difference-between-sdk-and-api/> (accessed Oct. 25, 2023).
- [11] “Lego admits it made too many bricks,” BBC News, <https://www.bbc.com/news/business-43298897> (accessed Sep. 14, 2023).
- [12] A. M. Bongale, DigitalCommons, Lincoln, Nebraska, rep., 2022
- [13] “Build new things from your good old bricks,” Brickit, <https://brickit.app/> (accessed Oct. 9, 2023).

- [14] “Sets,” Brickset.com, <https://brickset.com/sets#:~:text=Our%20database%20contains%20information%20on,over%20the%20last%2074%20years>. (accessed Apr. 10, 2024).
- [15] Bugsnag, “IOS performance monitoring best practices,” BugSnag, <https://www.bugsnag.com/blog/ios-performance-monitoring-best-practices/> (accessed Apr. 10, 2024).
- [16] E. Chan, “Most popular iPhone models of 2024: A user’s guide,” GadgetMates, <https://gadgetmates.com/most-popular-iphone#:~:text=Recent%20data%20indicates%20that%20the,have%20also%20seen%20significant%20sales>. (accessed Apr. 10, 2024).
- [17] P. Christensson, “Backend,” TechTerms, <https://techterms.com/definition/backend> (accessed Oct. 20, 2023).
- [18] P. Christensson, “Frontend,” TechTerms, <https://techterms.com/definition/frontend> (accessed Oct. 20, 2023).
- [19] P. Christensson, “Middleware,” TechTerms, <https://techterms.com/definition/middleware> (accessed Oct. 20, 2023).
- [20] Niels B. Christiansen, *The Lego Group Sustainability Progress*. Lego, Billund, Denmark, 2022 (accessed Oct. 20, 2023).
- [21] J. Diaz, “Everything you always wanted to know about Lego,” Gizmodo, <https://gizmodo.com/everything-you-always-wanted-to-know-about-lego-5019797> (accessed Sep. 14, 2023).
- [22] E. W. Dolan, “Spatial abilities help explain the positive association between Lego Skills and Mathematics Performance,” PsyPost, <https://www.psypost.org/2023/05/spatial-abilities-help-explain-the-positive-association-between-lego-skills-and-mathematics-performance-163201> (accessed Sep. 17, 2023).
- [23] “Object classification,” Object classification-ArcGIS Pro | Documentation, <https://pro.arcgis.com/en/pro-app/latest/tool-reference/image-analyst/object-classification.htm#:~:text=The%20goal%20of%20object%20classification,that%20contains%20the%20spectral%20bands> (accessed Oct. 25, 2023).
- [24] H. Enderton and R. R. Stoll, “Set theory,” Encyclopædia Britannica, <https://www.britannica.com/science/set-theory> (accessed Oct. 20, 2023).
- [25] “Client-server model,” GeeksforGeeks, <https://www.geeksforgeeks.org/client-server-model/> (accessed Sep. 21, 2023).
- [26] A. S. Gillis, “What is a thick client (fat client)? - definition from whatis.com,” WhatIs.com, <https://www.techtarget.com/whatis/definition/fat-client-thick-client> (accessed Nov. 11, 2023).

- [27] “Yolov5-Lite or Yolov5-nano? · issue #4462 · ultralytics/yolov5,” GitHub, <https://github.com/ultralytics/yolov5/issues/4462> (accessed Sep. 23, 2023).
- [28] “Model inference overview”, Google Cloud, <https://cloud.google.com/bigquery/docs/inference-overview#:~:text=Machine%20learning%20inference%20is%20the,machine%20learning%20model%20into%20production.%22> (accessed Oct. 20, 2023).
- [29] “Classification: True vs. false and positive vs. negative,” Google Developers, <https://developers.google.com/machine-learning/crash-course/classification/true-false-positive-negative#:~:text=A%20false%20positive%20is%20an,derived%20from%20these%20four%20outcomes> (accessed Oct. 20, 2023).
- [30] “What Are Convolutional Neural Networks?,” IBM, <https://www.ibm.com/topics/convolutional-neural-networks> (accessed Oct. 20, 2023).
- [31] “What is an Application Programming Interface (API)?,” IBM, <https://www.ibm.com/topics/api> (accessed Oct. 20, 2023).
- [32] “What is Artificial Intelligence (AI) ?,” IBM, <https://www.ibm.com/topics/artificial-intelligence> (accessed Oct. 20, 2023).
- [33] “What is Computer Vision?,” IBM, <https://www.ibm.com/topics/computer-vision> (accessed Oct. 20, 2023).
- [34] “What is Deep Learning?,” IBM, <https://www.ibm.com/topics/deep-learning> (accessed Oct. 20, 2023).
- [35] “What is Machine Learning?,” IBM, <https://www.ibm.com/topics/machine-learning#:~:text=Machine%20learning%20is%20a%20branch,learn%2C%20gradually%20improving%20its%20accuracy> (accessed Oct. 20, 2023).
- [36] “What is Natural Language Processing?,” IBM, <https://ibm.com/topics/natural-language-processing> (accessed Oct. 20, 2023).
- [37] “What Is a GPU? Graphics Processing Units Defined,” Intel, <https://www.intel.com/content/www/us/en/products/docs/processors/what-is-a-gpu.html> (accessed Oct. 20, 2023).
- [38] A. K, “Mastering transfer learning: A rock-paper-scissors case study,” AnalyticsVidhya.com, <https://www.analyticsvidhya.com/blog/2021/08/train-your-own-yolov5-object-detection-model/> <https://builtin.com/data-science/transfer-learning> (accessed Nov. 12, 2023).

- [39] T. Karmarkar, "Region proposal network (RPN) - backbone of faster R-CNN," Medium, <https://medium.com/@codeplumber/region-proposal-network-rpn-backbone-of-faster-r-cnn-4a744a38d7f9> (accessed Oct. 25, 2023).
- [40] Z. Keita, "How to measure model performance in Computer Vision: A comprehensive guide," Encord, <https://encord.com/blog/measure-model-performance-computer-vision/> (accessed Dec. 6, 2023).
- [41] "OFFICIAL LEGO® Shop CA," Lego, <https://www.lego.com/en-ca> (accessed Oct. 11, 2023).
- [42] Magnusson, "Implementing a Backup-Scheme with the 3-2-1 Strategy: A Comparison of the Active Solution with a New Implemented 3-2-1 Backup-Scheme", Mid Sweden University, tech., 2018 (accessed Sept. 25, 2023).
- [43] "What is object detection?," MATLAB & Simulink, <https://www.mathworks.com/discovery/object-detection.html#:~:text=Object%20detection%20is%20a%20computer,learning%20to%20produce%20meaningful%20results>. (accessed Oct. 9, 2023).
- [44] J. Olawanle, "What is a framework? ,," freeCodeCamp, <https://www.freecodecamp.org/news/what-is-a-framework-software-frameworks-definition/> (accessed Oct. 20, 2023).
- [45] "Distributed architecture," Online Tutorials, Courses, and eBooks Library, https://www.tutorialspoint.com/software_architecture_design/distributed_architecture.htm#:~:text=In%20distributed%20architecture%2C%20components%20are,a%20specific%20objective%20or%20goal. (accessed Oct. 25, 2023).
- [46] T. Pattinson, "Relational vs non-relational databases," Pluralsight, <https://www.pluralsight.com/blog/software-development/relational-vs-non-relational-databases> (accessed Oct. 2, 2023).
- [47] Quantization, https://huggingface.co/docs/optimum/concept_guides/quantization (accessed Oct. 25, 2023).
- [48] R. Sheldon, "What are sensors and how do they work?," TechTarget, <https://www.techtarget.com/whatis/definition/sensor> (accessed Oct. 20, 2023).
- [49] H. Singh, "Oracle Database Advantages, disadvantages and features [guide 2023]," The NineHertz, <https://theninehertz.com/blog/advantages-of-using-oracle-database> (accessed Sep. 23, 2023).
- [50] Simplilearn, "What is epoch in machine learning?," Simplilearn.com, <https://www.simplilearn.com/tutorials/machine-learning-tutorial/what-is-epoch-in-machine-learning#:~:text=An%20epoch%20is%20when%20all,dataset%20takes%20around%20a%20algorithm> (accessed Nov. 10, 2023).

- [51] E. Silva, “What is fuzzy matching?,” Redis, [https://redis.com/blog/what-is-fuzzy-matching/#:~:text=Fuzzy%20matching%20\(FM\)%2C%20also,elements%20in%20data%20table%20sets](https://redis.com/blog/what-is-fuzzy-matching/#:~:text=Fuzzy%20matching%20(FM)%2C%20also,elements%20in%20data%20table%20sets). (accessed Oct. 9, 2023).
- [52] “Power analysis in statistics: Sample Size, significance, and statistical power,” Spotfire, <https://www.spotfire.com/glossary/what-is-power-analysis#:~:text=A%20power%20analysis%20is%20the,it%20is%20genuine%20and%20significant> (accessed Apr. 10, 2024)
- [53] S. Sutherland, “Lego bricks could survive in the ocean for more than 1,000 years,” The Weather Network, <https://www.theweathernetwork.com/en/news/lifestyle/travel/plastic-lego-bricks-could-survive-in-the-ocean-for-up-to-1300-years> (accessed Sep. 15, 2023).
- [54] “What is gyroscope?,” TechTarget, <https://www.techtarget.com/whatis/definition/gyroscope> (accessed Oct. 20, 2023).
- [55] “What is event-driven architecture?,” TIBCO Software, [https://www.tibco.com/reference-center/what-is-event-driven-architecture#:~:text=Event%2Ddriven%20architecture%20\(EDA\),time%20or%20near%20real%20time](https://www.tibco.com/reference-center/what-is-event-driven-architecture#:~:text=Event%2Ddriven%20architecture%20(EDA),time%20or%20near%20real%20time). (accessed Sep. 24, 2023).
- [56] “How WCAG 2.0 and Other W3C/WAI Guidelines Apply to Mobile,” W3C, <https://www.w3.org/TR/mobile-accessibility-mapping/>
- [57] “MySQL Advantages and disadvantages,” W3schools, , <https://www.w3schools.blog/mysql-advantages-disadvantages> (accessed Sep. 24, 2023).
- [58] “Weights and biases in machine learning,” Weights and Biases in machine learning | H2O.ai Wiki, <https://h2o.ai/wiki/weights-and-biases/#:~:text=Weights%20set%20the%20standards%20for,network%20did%20not%20previously%20have> (accessed Nov. 12, 2023).

Appendix 1 Glossary

Glossary

Accelerometer	A sensor that measures the acceleration of an object it is attached to. In smartphones it is used to detect changes in motion or orientation. [43]
Agile Methodology	An iterative and flexible approach to software development and project management, Emphasizing collaboration, client feedback, and incremental delivery. [8]
Application Programming Interfaces (API)	A set of rules or protocols that allow different software applications to communicate with each other. Often developed by teams and allowed for public use via API keys which can be free or not. [31]
Artificial Intelligence	The development of computer systems and algorithms that can perform tasks typically requiring human intelligence. Used as an umbrella term for ideas such as “deep learning,” “machine learning,” and “computer vision.” [32]
Backend	The server side of an application. Responsible for managing and processing data, business logic, and database interactions. [17]
Computer Vision	A field of Artificial Intelligence that focuses on enabling computers to interpret and understand visual information from images and videos. [33]
Convolutional Neural Network	A type of neural network designed specifically for processing data in grid-like structure, such as images and videos. [30]
Deep Learning	A subfield of machine learning that involves artificial neural networks, capable of automatically learning and extracting hierarchical features from data. [34]
Distributed Architecture	Components are located on different platforms and several components can work together across a communication network. [45]
Epochs	Represents one complete pass through the entire dataset during training. Initially, the more epochs we train a model on, the more it learns, and the better it performs [50]

Event Driven Architecture	An architecture that uses events to communicate between decoupled services. When an event occurs a different aspect of a program is triggered/activated. [9]
False Negative	When a model incorrectly predicts a negative outcome from a data point that is in reality positive. The opposite of a false positive. [29]
False Positive	When a model incorrectly predicts a positive outcome from a data point that is in reality negative. An error where the model mistakenly identifies data as belonging to one category. [29]
Fat Client	A software application that performs most of its significant processing and data management on the client side. [26]
Framework	A pre-established structure or set of tools and libraries that provide a foundation for building applications. It offers a standardized way to design and develop software by defining and enforcing common practices. [44]
Frontend	The user interface and “presentation” layer of a software application. Where the design, layout, and interactive components of an application live. [18]
Fuzzy Matching	A technique used in machine learning to compare and match data with a degree of flexibility, allowing for approximate matches rather than perfect ones [46].
Graphical Processing Unit (GPU)	A specialized computer chip used for rendering and manipulating graphics and images. Recently GPUs have gained popularity in high-performance computing tasks like those in machine learning. [34]
Gyroscope	A device that measures orientation and angular velocity using principles of angular momentum. It is used in navigation systems, aerospace applications, and various electronic devices like mobile phones. [48]
Inference	The process of using a trained machine learning model to make predictions based on new, unseen data. [28]

Kanban Methodology	An Agile framework visualization methodology. It emphasizes visualizing work as it flows through its life cycle by using boards with columns to show where in the workflow a task is and cards to describe each task. [8]
Machine Learning	A branch of Artificial Intelligence with the goal of using algorithms to emulate human learning and improve their performance on specific tasks by learning from data, without explicit programming. [35]
Middleware	The intermediate layer between frontend and backend. Facilitates communication and data exchange between these two components. [19]
Natural Language Processing	A field of Artificial Intelligence that focuses on the interaction between computers and human language. It involves the development of algorithms to enable computers to understand, interpret, and generate human language. [36]
Object Classification	Used to determine and categorize objects based on their features. [23]
Quantizing	A machine learning technique to reduce the number of bits used to represent the weights, activations, or parameters of a neural network model. This compression technique is used to make the model more memory-efficient and faster to execute, often at the cost of some loss in precision. [47]
Region Proposal Generation	A protocol that is with computer vision to assist in finding objects or regions in an image that are of interest. [39]
Scrum Framework	An Agile framework for structured incremental delivery for managing complex projects. Involves dividing work into small, manageable iterations called “sprints”. [8]
Set Theory	A branch of mathematics that deals with collections of data, called sets. It involves exploring the relationships, operations, and logic of sets. [22]
Software Development Kit	A collection of tools, libraries, and documentation that help developers create applications for specific platforms or devices more efficiently. [10]
Sprint	In the context of Scrum and Agile project management, a sprint is a time-boxed, fixed-duration period (typically 2 to 4

weeks) during which a development team works to complete a set of predefined tasks or user stories. [8]

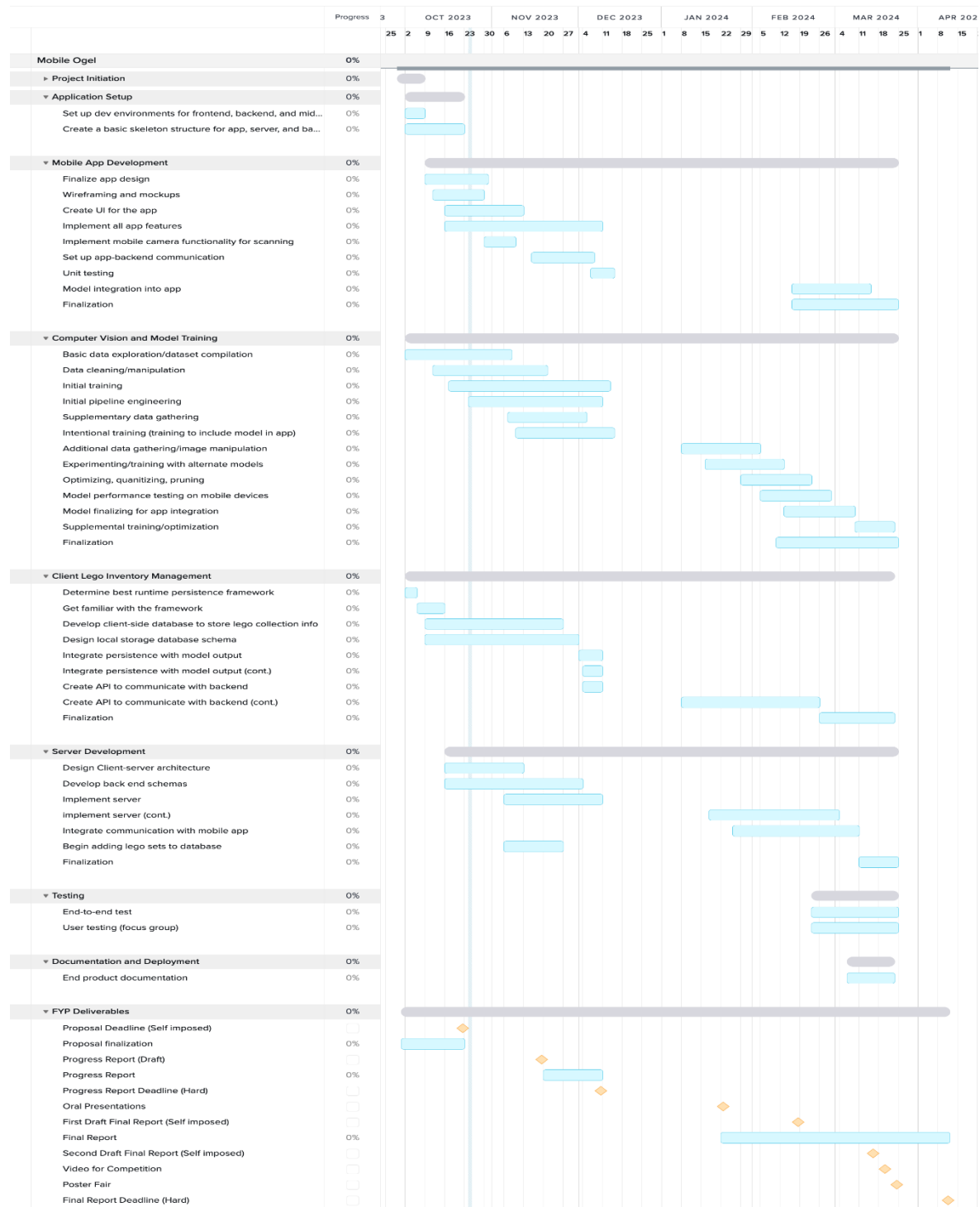
Weights

Refers to the parameters of the model. The weight value determines the influence input data has on the output produced [58]

YOLO

A popular real-time object detection computer vision model. It uses 1-stage architecture which is where the acronym YOLO comes from, standing for “You only look once” [27].

Appendix 2 Gantt Chart



https://github.com/mobileogel/MobileOgelUI/blob/ffa5af7e5ebb859ea293bb3f5390061ede4b51fc/gantt_progress_report_final.png

Appendix 3 Course Codes

CCDP 2100 – Communication Skills for Engineering Students
COMP 3005 – Database Management Systems
COMP 3008 – Human-Computer Interaction
COMP 4102 – Computer Vision
SYSC 2004 – Object-Oriented Software Development
SYSC 2006 – Foundations of Imperative Programming
SYSC 2100 – Algorithms and Data Structures
SYSC 2320 – Introduction to Computer Organization and Architecture
SYSC 3110 – Software Development Project
SYSC 3120 – Software Requirements Engineering
SYSC 3310 – Introduction to Real-Time Systems
SYSC 4106 – The Software Economy and Project Management
SYSC 4120 – Software Architecture and Design
SYSC 4130 – Human Computer Interaction
SYSC 4415 – Introduction to Machine Learning
SYSC 4504 – Fundamentals of Web Development
SYSC 4806 – Software Engineering Lab

Appendix 4 Progress Report

Fall Progress Report and Project Proposal will be attached below (pg 87 onwards)